



**Escuela Técnica Superior de Ingeniería  
Universidad de Huelva**

Máster en Ingeniería Informática

**Trabajo Fin de Máster**

*DeepSexist: Detección y Categorización de Mensajes  
Sexistas en Vídeos Cortos Mediante Deep Learning*

Adrián Moreno Monterde  
septiembre, 2026

*A ChatGPT*  
*Por estar siempre a mi lado mientras hacía este TFG/TFM*

# Resumen

---

Este apartado es muy importante, ya que es lo primero que se lee. Su redacción debe ser muy correcta y reflejar, de forma sintética, todo el trabajo realizado.

En una página como máximo, el resumen explica de modo conciso lo que intenta resolver el trabajo (¿Qué?), la metodología para abordar su solución (¿Cómo?), así como los principales resultados y conclusiones del trabajo.

**Palabras clave:** listado de palabras clave, separadas por coma, que representan el trabajo

# Abstract

---

This section is very important, since it is the first thing that is read. Its wording must be very correct and reflect, in a synthetic way, all the work done.

In a maximum of one page, the abstract explains in a concise way what the work tries to solve (What?), the methodology to approach its solution (How?), as well as the main results and conclusions of the work.

**Keywords:** list of keywords, separated by comma, representing the work

# Agradecimientos

---

Aunque no es un apartado obligatorio, dedicar unas líneas a expresar agradecimiento es una buena forma de reconocer el apoyo recibido a lo largo del proceso. Este espacio permite dar las gracias a todas aquellas personas que, de una u otra manera, han contribuido a que este trabajo haya podido desarrollarse y finalizar con éxito. Es habitual mencionar aquí a tutores, profesores, familiares, compañeros o amistades que hayan acompañado durante el camino.

Los agradecimientos pueden tener un tono personal e incluir alguna anécdota si se desea, pero se recomienda que no excedan una página de extensión.

*Nombre del autor*

Huelva, 2025

# Índice general

---

|                                                                          |          |
|--------------------------------------------------------------------------|----------|
| Resumen                                                                  | II       |
| Abstract                                                                 | III      |
| Agradecimientos                                                          | IV       |
| Índice de Figuras                                                        | VII      |
| Índice de Tablas                                                         | VIII     |
| <b>1. Introducción</b>                                                   | <b>1</b> |
| 1.1. Motivación                                                          | 1        |
| 1.2. Objetivos                                                           | 2        |
| 1.3. Competencias                                                        | 3        |
| 1.4. Estructura de la Memoria                                            | 4        |
| <b>2. Marco Teórico</b>                                                  | <b>6</b> |
| 2.1. Aprendizaje Automático                                              | 6        |
| 2.2. Clasificación de textos, imágenes y vídeos                          | 7        |
| 2.3. Aprendizaje Profundo                                                | 8        |
| 2.3.1. Epochs y tamaño de lote                                           | 10       |
| 2.3.2. Tasa de aprendizaje y <i>weight decay</i>                         | 11       |
| 2.3.3. Función de pérdida                                                | 12       |
| 2.3.4. Overfitting y técnicas de regularización                          | 12       |
| 2.3.5. Early stopping y validación                                       | 13       |
| 2.4. Procesamiento del Lenguaje Natural                                  | 14       |
| 2.4.1. Preprocesamiento del texto                                        | 15       |
| 2.4.2. Representación del texto                                          | 15       |
| 2.4.3. Clasificación de texto en PLN                                     | 16       |
| 2.5. Modelos Avanzados para Texto                                        | 17       |
| 2.5.1. Transformers                                                      | 17       |
| 2.5.2. Modelos Generativos de Lenguaje ( <i>LLMs</i> )                   | 20       |
| 2.6. Modelos Avanzados para Visión Estática (Imágenes)                   | 22       |
| 2.6.1. <i>Vision Transformers</i> ( <i>ViT</i> ) y variantes jerárquicas | 22       |
| 2.6.2. Redes Neuronales Convolucionales Modernas ( <i>ConvNeXt</i> )     | 23       |
| 2.6.3. Agregación temporal para clasificación de vídeo                   | 24       |
| 2.7. Modelos Avanzados para Visión Temporal (Vídeos)                     | 24       |
| 2.7.1. Adaptación de la Arquitectura <i>Transformer</i> al Vídeo         | 25       |
| 2.7.2. Enfoques Basados en Autoencoders Enmascarados                     | 26       |
| 2.8. Aprendizaje por Transferencia                                       | 27       |
| 2.8.1. Fine-tuning y optimización de hiperparámetros                     | 27       |
| 2.9. Ensemble de Modelos y Fusión multimodal                             | 29       |
| 2.9.1. Ventajas de los <i>Transformers</i> en tareas multimodales        | 29       |
| 2.9.2. Fusión Temprana vs Fusión Tardía                                  | 29       |
| 2.10. Subjetividad en la anotación de los datos                          | 30       |
| 2.10.1. Aprendizaje con Desacuerdo (Learning with Disagreement)          | 31       |

|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| 2.10.2. Perspectivismo en Inteligencia Artificial . . . . .                    | 31        |
| 2.11. Medidas de Evaluación . . . . .                                          | 32        |
| 2.11.1. <i>Accuracy</i> , Precisión, Exhaustividad y <i>F1-Score</i> . . . . . | 32        |
| 2.11.2. Evaluación en clasificación multietiqueta . . . . .                    | 33        |
| 2.11.3. Métricas para evaluación con desacuerdo . . . . .                      | 34        |
| 2.12. Tecnologías y Recursos Utilizados . . . . .                              | 34        |
| <b>3. Metodología, Experimentación y Resultados</b>                            | <b>37</b> |
| 3.1. Participación en una tarea (si es el caso) . . . . .                      | 37        |
| 3.2. Descripción General de la Metodología . . . . .                           | 38        |
| 3.3. Preparación de los Datos . . . . .                                        | 39        |
| 3.3.1. Descripción de los Datos . . . . .                                      | 40        |
| 3.3.2. Balanceo de Datos . . . . .                                             | 40        |
| 3.3.3. Limpieza y Normalización . . . . .                                      | 41        |
| 3.3.4. Tokenización y Codificación . . . . .                                   | 41        |
| 3.3.5. División de Datos . . . . .                                             | 41        |
| 3.4. Selección de los Modelos . . . . .                                        | 42        |
| 3.5. Ajuste de Hiperparámetros . . . . .                                       | 42        |
| 3.5.1. Métodos de Búsqueda . . . . .                                           | 42        |
| 3.5.2. Hiperparámetros Evaluados . . . . .                                     | 43        |
| 3.5.3. Tamaño del Conjunto de Datos Utilizado . . . . .                        | 43        |
| 3.5.4. Criterios de Selección . . . . .                                        | 43        |
| 3.6. Entrenamiento de los Modelos . . . . .                                    | 43        |
| 3.6.1. Baselines . . . . .                                                     | 44        |
| 3.6.2. Configuración de Entrenamiento . . . . .                                | 44        |
| 3.6.3. Monitorización del Entrenamiento . . . . .                              | 45        |
| 3.7. Evaluación y Análisis de Resultados . . . . .                             | 45        |
| 3.8. Análisis de Errores . . . . .                                             | 48        |
| 3.9. Resultados Oficiales de la Tarea (opcional) . . . . .                     | 50        |
| <b>4. Conclusiones y Trabajo Futuro</b>                                        | <b>52</b> |
| 4.1. Conclusiones . . . . .                                                    | 52        |
| 4.2. Trabajo Futuro . . . . .                                                  | 53        |
| 4.3. Planificación Temporal del Trabajo Realizado . . . . .                    | 54        |
| <b>Referencias</b>                                                             | <b>56</b> |
| <b>A. Anexos</b>                                                               | <b>60</b> |
| A.1. Aplicación Web para la Visualización y Prueba del Modelo . . . . .        | 60        |

# Índice de Figuras

---

|       |                                                                              |    |
|-------|------------------------------------------------------------------------------|----|
| 2.1.  | Flujo de procesamiento multimodal de vídeos (texto + imagen + vídeos) . .    | 8  |
| 2.2.  | Principal diferencia entre <i>Machine</i> y <i>Deep Learning</i> . . . . .   | 9  |
| 2.3.  | Arquitectura de los modelos <i>Transformers</i> . . . . .                    | 18 |
| 2.4.  | Ejemplo de funcionamiento del mecanismo de atención . . . . .                | 19 |
| 2.5.  | Entradas necesarias para el codificador de <i>BERT</i> . . . . .             | 19 |
| 2.6.  | Arquitectura de un <i>ViT</i> . . . . .                                      | 23 |
| 2.7.  | Ilustración de los mecanismos de atención de un <i>TimeSformer</i> . . . . . | 25 |
| 2.8.  | Representación de los <i>Tubelets</i> de los <i>ViViT</i> . . . . .          | 26 |
| 2.9.  | Arquitectura de <i>VideoMAE</i> . . . . .                                    | 27 |
| 2.10. | Tecnologías usadas para el desarrollo del TFM . . . . .                      | 35 |
| 3.1.  | Pipeline general del marco de experimentación. . . . .                       | 39 |
| 3.2.  | Evolución del entrenamiento monitorizada con TensorBoard. . . . .            | 45 |
| 3.3.  | Matriz de confusión para el modelo RoBERTa. . . . .                          | 47 |
| 3.4.  | Matriz de confusión para el modelo RoBERTa. . . . .                          | 47 |
| 3.5.  | Curvas ROC de los modelos evaluados. . . . .                                 | 48 |



# Índice de Tablas

---

|                                                                                  |    |
|----------------------------------------------------------------------------------|----|
| 3.1. Resultados de evaluación sobre el conjunto de test. . . . .                 | 46 |
| 3.2. Resultados por clase para los modelos evaluados (conjunto de test). . . . . | 46 |
| 3.3. Resultados oficiales publicados por la organización de la tarea. . . . .    | 51 |
| 4.1. Planificación temporal del trabajo realizado. . . . .                       | 54 |

# Introducción

---

Todo proyecto de investigación necesita un contexto claro que justifique su desarrollo. Esta memoria recoge el trabajo realizado durante nuestra participación en la competición internacional *EXIST 2025*, cuyo reto principal es la detección y categorización automática del sexismo en redes sociales a partir del análisis combinado de texto, imágenes y vídeos.

A lo largo de este capítulo introductorio vamos a desglosar los motivos que nos han llevado a elegir este tema, destacando la necesidad urgente de aplicar la Inteligencia Artificial para frenar la discriminación y el odio en las redes sociales. Además, definiremos los objetivos concretos que han marcado el rumbo técnico del proyecto y repasaremos las competencias académicas adquiridas como frutos de la investigación. Como cierre, se presenta un breve itinerario al lector para guiarlo a través de los capítulos posteriores: el marco teórico, la metodología aplicada, la fase de experimentación y las conclusiones finales.

### 1.1. Motivación

El uso masivo de las plataformas sociales ha democratizado la comunicación, pero al mismo tiempo ha facilitado la propagación a gran escala del discurso de odio, debido al anonimato y, en la mayoría de los casos, impunidad. En este contexto digital, el sexismo destaca como un problema estructural que afecta de manera desproporcionada a las mujeres [1]. En redes basadas en texto como *Twitter* (actualmente *X*), la misoginia se hace patente mediante el acoso y la creación de narrativas discriminatorias, mientras que en plataformas de consumo rápido como *TikTok*, las dinámicas de los algoritmos pueden amplificar ideologías machistas y fomentar la hipersexualización. Paralelamente, el uso de formatos multimodales, como los memes, permite camuflar los prejuicios bajo la apariencia de humor o ironía, convirtiéndolos en mecanismos de difusión muy escurridizos.

Uno de los grandes retos para frenar este fenómeno es que el sexismo no siempre se manifiesta de forma explícita o mediante el uso de palabras malsonantes. Frecuentemente, se apoya en estereotipos profundamente arraigados, sesgos implícitos y microagresiones que normalizan la desigualdad. Comprender y detectar estas formas sutiles es crucial para abordar el problema, ya que su impunidad influye negativamente sobre la percepción social y perpetúa la discriminación. Sin embargo, la revisión y moderación manual de este tipo de contenido resulta en la actualidad completamente inviable, debido al inmenso volumen

de publicaciones que se generan constantemente, lo cual supera con creces la capacidad humana [2].

Por todo ello, resulta indispensable investigar y desarrollar herramientas automatizadas capaces de entender estas dinámicas tan complejas. Mediante este trabajo, proponemos aplicar y evaluar diversas técnicas de *Deep Learning* para construir modelos mediante un abordaje multimodal que ayuden a clasificar las publicaciones en las redes sociales. El propósito no es únicamente detectar la presencia de sexismo, sino caracterizarlo en función de la intención del mensaje y la categoría del abuso. Con la implementación de estas técnicas, buscamos proporcionar mecanismos escalables que apoyen la moderación de contenido, ayudando a mitigar la violencia de género en la red y contribuyendo a lograr plataformas digitales más inclusivas y libres de abusos.

## 1.2. Objetivos

El objetivo principal de este proyecto es diseñar, implementar y evaluar un sistema automatizado basado en técnicas de aprendizaje profundo (*Deep Learning*) para la detección, en la Tarea 3.1, y categorización del sexismo, en la Tarea 3.3, en vídeos de la plataforma *TikTok*. Para lograr este propósito, hemos planteado un enfoque multimodal capaz de procesar tanto de forma combinada como de manera independiente la información textual, a partir de las transcripciones de los vídeos, y la información visual y espaciotemporal, mediante la extracción y análisis de sus fotogramas (*frames*), enmarcando la investigación dentro de la competición internacional *EXIST 2025*.

Para alcanzar dicho objetivo general, se han definido los siguientes objetivos específicos:

- **Estudiar** el estado del arte y las técnicas actuales en el campo del *Deep Learning*, el Procesamiento del Lenguaje Natural (PLN) y la Visión Artificial aplicadas a la detección de contenido de odio y sexismo en formato vídeo.
- **Preprocesar y adecuar** el conjunto de datos proporcionado por la organización de EXIST 2025. Esto incluye el procesamiento de las transcripciones textuales y el diseño de un canal de extracción de fotogramas para posibilitar el análisis visual, gestionando además el paradigma *Learning with Disagreement* (LeWiDi).
- **Implementar y ajustar (*fine-tuning*)** modelos de estado del arte especializados en las diferentes modalidades trabajadas a partir del vídeo, contrastando arquitecturas de lenguaje con arquitecturas de visión estática y temporal.
- **Proponer y desarrollar** estrategias de fusión de modelos (*Ensemble*) para combinar las predicciones de las modalidades de texto, imagen y vídeo, con el fin de incrementar el rendimiento general y mitigar las deficiencias individuales de cada modelo.
- **Participar** en la tarea 3.1 de la competición *EXIST 2025*, estableciendo líneas base

de rendimiento mediante el uso de modelos de lenguaje supervisados (*RoBERTa*) y enfoques *zero-shot* mediante Grandes Modelos de Lenguaje (*LLaMA*).

- **Desarrollar**, en una fase de experimentación posterior a la competición, un marco de evaluación exhaustivo para las tareas 3.1 y 3.3, optimizando y fusionando (*Ensemble*) los mejores modelos de cada modalidad (texto, imagen y vídeo) para maximizar el rendimiento general.
- **Explorar** el impacto del perspectivismo y la subjetividad en la detección del sexismo, evaluando cómo el género de los anotadores influye en el sesgo del entrenamiento de los modelos.

### 1.3. Competencias

La realización de este Trabajo de Fin de Máster ha permitido poner en práctica y consolidar una serie de competencias fundamentales adquiridas durante la titulación, adaptándolas a un problema de investigación real y de alto impacto social.

De acuerdo con lo establecido en el documento de propuesta (Anexo II), la **competencia principal** que rige este trabajo es:

- **Capacidad de aplicar métodos matemáticos, estadísticos y de inteligencia artificial para modelar, diseñar y desarrollar aplicaciones, servicios, sistemas inteligentes y sistemas basados en el conocimiento.** Esta competencia se ha alcanzado de manera integral mediante el diseño y desarrollo de un sistema inteligente multimodal. Para ello, se han aplicado métodos avanzados de inteligencia artificial (*Deep Learning*) y estrategias estadísticas (como la agregación de distribuciones de probabilidad en etiquetas *soft* y el diseño de *ensembles*) con el fin de modelar y detectar el comportamiento sexista en la plataforma *TikTok*.

Además de esta competencia principal, el carácter de investigación de este proyecto nos ha permitido alcanzar las siguientes competencias adicionales vinculadas al plan de estudios:

- **Capacidad para el análisis, procesamiento y extracción automática de información a partir de datos no estructurados y multimodales.** Se ha logrado mediante la construcción de un flujo de trabajo técnico capaz de extraer fotogramas (*frames*) secuenciales a partir de los archivos de vídeo, así como trabajar con las transcripciones textuales de estos. Esto ha implicado limpiar el ruido implícito del lenguaje de las redes sociales y estructurar información bajo el complejo paradigma de las anotaciones múltiples (*Learning with Disagreement*).
- **Capacidad para conocer, diseñar y entrenar modelos *Deep Learning* aplicados a Visión Artificial y Procesamiento del Lenguaje Natural.** Esta capa-

cidad se ha consolidado a través de la implementación, adaptación y optimización de diversas arquitecturas expuestas en el capítulo del estado del arte. Entre ellas destacan modelos de lenguaje, como los *Transformers* y *Large Language Model (LLMs)*, y redes avanzadas de visión estática espaciotemporal, como los *Vision Transformers (ViT)* y *Video Vision Transformers (ViViT)*.

- **Capacidad para evaluar sistemas complejos y adquirir conocimientos a partir de grandes volúmenes de datos.** Esta competencia se ha desarrollado al enfrentar problemas de modelo del mundo real, como el desbalanceo de clases y colapso modal (*Mode Collapse*). La validación del sistema se ha fundamentado en la interpretación de métricas de evaluación avanzadas, como el *Information Contrast Measure (ICM)* o el *F1-score* para determinar los mejores rendimientos de los modelos y, por ende, poder establecer las mejores estrategias de fusión de modelos.
- **Conciencia ética y capacidad para analizar el impacto social y los sesgos en los sistemas de Inteligencia Artificial.** Se ha adquirido de manera transversal al explorar el perspectivismo y la subjetividad en la detección del sexismo. El análisis de algunos de los factores de los anotadores (como el género) pueden influir en el sesgo de entrenamiento de los modelos, permitiéndonos comprender la importancia crítica de diseñar sistemas de expertos más justos, transparentes y representativos.

## 1.4. Estructura de la Memoria

Los siguientes capítulos de la memoria de estructuran de la siguiente manera:

- En el [Capítulo 2](#), Marco Teórico, se exponen los conceptos fundamentales que sustentan este trabajo. Esto incluye los principios del *Deep Learning* y el Procesamiento del Lenguaje Natural, así como las arquitecturas de los modelos de lenguaje y de los modelos de clasificación de imagen y vídeo. También se describen técnicas clave como el aprendizaje por transferencia, estrategias de combinación de modelos, el paradigma del aprendizaje con desacuerdo y las métricas de evaluación empleadas.
- En el [Capítulo 3](#), Metodología, Experimentación y Resultados, se describe la metodología propuesta para alcanzar los objetivos planteados. Se detalla la participación en la competición *EXIST 2025* asociada a este trabajo, así como la preparación de los datos y las técnicas implementadas durante el ajuste de hiperparámetros y el entrenamiento de los modelos. Finalmente, se muestran y analizan los resultados obtenidos, incluyendo un análisis de errores y la exploración del perspectivismo.
- En el [Capítulo 4](#), Conclusiones y Trabajo Futuro, se presentan las conclusiones finales extraídas tras la realización de este proyecto, evaluando el grado de cumplimiento de los objetivos. Del mismo modo, se plantean las posibles directrices para investigaciones futuras y se detalla la planificación temporal seguida durante el desarrollo

del trabajo.

# Marco Teórico

---

En este capítulo se describen los conceptos teóricos y el estado del arte necesarios para comprender el desarrollo y las decisiones de arquitectura tomadas para la realización de este Trabajo de Fin de Máster.

Dada la naturaleza compleja y multimodal del problema abordado (la detección y categorización del sexismo en redes sociales), el marco teórico abarca desde los principios fundamentales del Aprendizaje Automático y Profundo (*Deep Learning*) hasta los enfoques más modernos en Procesamiento del Lenguaje Natural y Visión Artificial. A lo largo de las siguientes secciones, se realiza una revisión crítica de las arquitecturas de estado del arte, prestando especial atención a los modelos basados en Transformers y a los Grandes Modelos de Lenguaje (*LLMs*). Asimismo, se fundamentan de manera teórica las estrategias empleadas para la experimentación, como el aprendizaje por transferencia, la fusión de modelos (*ensembles*) y el novedoso paradigma del aprendizaje con desacuerdo (*LeWiDi*), justificando su idoneidad para modelar la subjetividad y el sesgo en las plataformas digitales.

## 2.1. Aprendizaje Automático

El aprendizaje automático, comúnmente conocido como *Machine Learning (ML)*, es una de las disciplinas fundamentales dentro de la Inteligencia Artificial. Su objetivo principal es el desarrollo de algoritmos y modelos computacionales capaces de identificar patrones complejos y adquirir conocimiento empírico a partir de los datos, permitiendo a los sistemas mejorar su rendimiento en una tarea específica sin necesidad de ser explícitamente programados mediante reglas deterministas [3].

De manera clásica, atendiendo a la naturaleza de los datos de entrenamiento y al tipo de retroalimentación que recibe el sistema, distinguimos entre tres tipos de aprendizaje [4]:

- Aprendizaje Supervisado: En este enfoque, el algoritmo se entrena utilizando un conjunto de datos etiquetado, es decir, cada dato de entrada está emparejado de antemano con su resultado correcto o salida esperada. El objetivo del modelo es aprender una función matemática subyacente que relacione de forma generalizada las entradas con las salidas. Una vez entrenado, el sistema es capaz de inferir y predecir resultados

con cierto grado de precisión frente a nuevos datos con los que no ha interactuado previamente [5].

- Aprendizaje No Supervisado: A diferencia del tipo de aprendizaje anterior, en este caso el modelo recibe un conjunto de datos que carece por completo de etiquetas o salidas conocidas. El propósito del algoritmo no es predecir un valor en concreto, sino explorar la información para descubrir estructuras, patrones o agrupaciones de manera autónoma.
- Aprendizaje por Refuerzo: En este escenario no se parte de un conjunto de datos estático, sino que un agente aprende a tomar secuencias de decisiones interactuando dinámicamente con un entorno. Mediante un proceso continuo de prueba y error, el modelo ajusta su comportamiento con el objetivo de maximizar una señal de recompensa a largo plazo [6].

Para el desarrollo del presente trabajo, el enfoque central se enmarca dentro del Aprendizaje Supervisado. Dado que el objetivo es la detección y categorización del sexismo en la plataforma *TikTok*, los modelos requieren ser entrenados con colecciones de datos previamente anotados que asocian cada publicación (ya sea el vídeo, la transcripción textual o sus fotogramas) con una clase específica. Sin embargo, dada la alta subjetividad inherente a la interpretación de los discursos de odio en redes sociales, el concepto tradicional de “etiqueta única” del aprendizaje supervisado clásico presenta limitaciones, lo cual motivará la adopción de enfoques más flexibles como el aprendizaje con desacuerdo, que se abordará en secciones posteriores.

## 2.2. Clasificación de textos, imágenes y vídeos

La clasificación automática es una de las tareas fundamentales del aprendizaje automático supervisado. Consiste en asignar una o varias categorías a un dato de entrada a partir de un conjunto de ejemplos que estaban previamente etiquetados. Atendiendo a la cantidad de categorías y a su exclusividad, los problemas de clasificación pueden dividirse en:

- Clasificación binaria: Cuando el sistema debe discriminar entre dos únicas clases (0 o 1).
- Clasificación multiclase: Cuando el sistema debe discriminar entre más de dos clases, siendo éstas discriminantes entre sí. Esto quiere decir que dado un dato de entrada, este puede pertenecer a una de las clases sobre las que se esté clasificando.
- Clasificación multietiqueta: Cuando el sistema debe discriminar entre más de dos clases, siendo éstas no discriminantes entre sí. Esto quiere decir que una misma entrada puede no pertenecer a ninguna clase, pertenecer a una exclusivamente o pertenecer a varias simultáneamente [7].



Tradicionalmente, los modelos de clasificación se han diseñado para procesar una única modalidad de datos. En la clasificación de texto, la entrada consiste en secuencias de caracteres o palabras, requiriendo técnicas de Procesamiento de Lenguaje Natural para extraer la semántica, la sintaxis y el contexto del mensaje [8]. En la clasificación de imágenes, los algoritmos analizan la distribución espacial de los píxeles para identificar formas, texturas y patrones visuales estáticos. Por su parte, la clasificación de vídeo introduce una dimensión adicional y crítica: el tiempo. El procesamiento de este tipo de datos requiere no solo comprender la información visual estática de cada fotograma (*frame*), sino también modelar el movimiento y la evolución espaciotemporal a lo largo de la secuencia [9].

No obstante, en entornos reales donde la información es abundante y compleja, el uso de una modalidad suele ser insuficiente. Esto ha impulsado el desarrollo de la clasificación multimodal, un paradigma que integra múltiples fuentes de información de distinta naturaleza (por ejemplo, combinando un texto con una secuencia de imágenes). Para lograrlo, estos sistemas emplean arquitecturas especializadas en cada dominio y, posteriormente, aplican estrategias de fusión que combinan las representaciones extraídas, permitiendo al modelo tomar una decisión conjunta mucho más robusta frente a la ambigüedad [10].

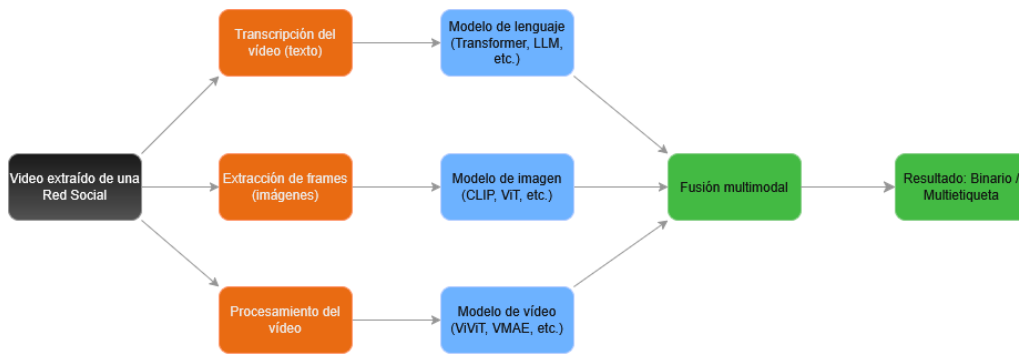


Figura 2.1: Flujo de procesamiento multimodal de vídeos (texto + imagen + vídeos)

Como se ilustra de manera conceptual en la Figura 2.1, un flujo de procesamiento multimodal estándar divide el contenido original en sus componentes básicos. Cada modalidad es procesada de forma independiente y en paralelo por modelos específicos para extraer sus características intrínsecas. Finalmente, estas representaciones convergen en un módulo de fusión, el cual combina la información para emitir la predicción final. Este esquema teórico sirve como pilar fundamental para el diseño de arquitecturas capaces de analizar el complejo contenido multimedia actual.

## 2.3. Aprendizaje Profundo

Dentro de la disciplina del aprendizaje automático, el aprendizaje profundo (*Deep Learning*) representa una evolución significativa [11]. Esta rama se fundamenta en el uso de

redes neuronales artificiales, arquitecturas computacionales inspiradas en cómo el cerebro humano estructura y procesa la información. Al organizar estas neuronas artificiales en múltiples capas sucesivas, el sistema adquiere la capacidad de procesar la información de forma jerárquica, logrando un nivel de abstracción cada vez mayor. A medida que aumenta la profundidad de la red (el número de capas), el algoritmo es capaz de modelar relaciones más complejas y extraer mayor cantidad de información a partir de los datos, incluso si no es perceptible por el ser humano [12].

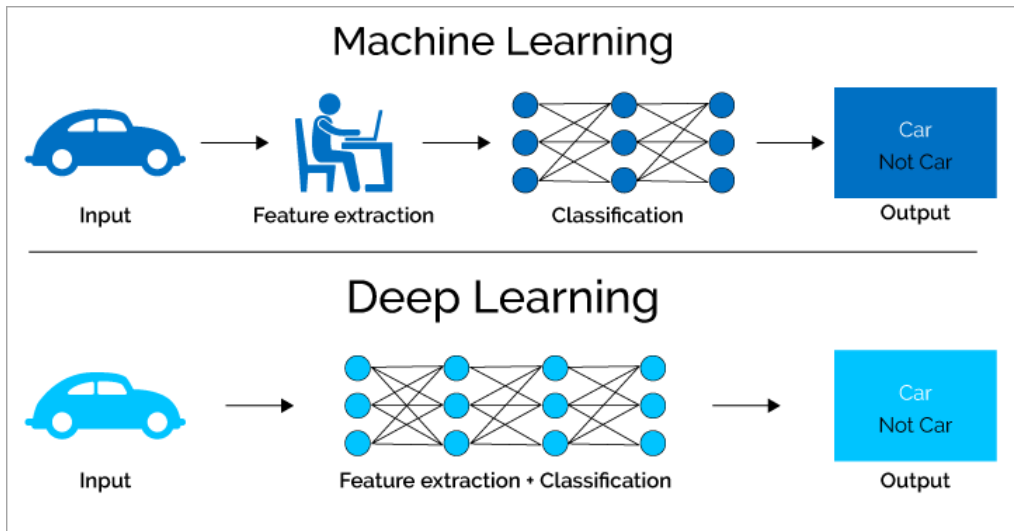


Figura 2.2: Principal diferencia entre *Machine* y *Deep Learning*

La principal diferencia entre el aprendizaje automático clásico y el aprendizaje profundo radica en el proceso de extracción de características (*feature extraction*). Como se ilustra en la Figura 2.2, en los algoritmos tradicionales de *Machine Learning*, es necesario que un experto humano o un proceso algorítmico previo diseñe y extraiga manualmente las características relevantes de los datos de entrada para que el clasificador pueda tomar decisiones. Por el contrario, los modelos de *Deep Learning* automatizan este proceso por completo, integrando la extracción de características de forma implícita dentro de la propia arquitectura de la red. Esta capacidad de aprendizaje de extremo a extremo (*end-to-end*) permite al modelo descubrir por sí mismo qué patrones son los más instructivos para resolver la tarea[13].

En el contexto de este Trabajo de Fin de Máster, el aprendizaje profundo constituye el núcleo técnico del sistema propuesto. Se han empleado modelos basados en arquitecturas profundas, tanto para el procesamiento de la información textual, con modelos *Transformers* y Grandes Modelos de Lenguaje, como para el análisis visual, con modelos *Vision Transformers*.

Dado el enorme tamaño de estas arquitecturas y la complejidad matemática que entraña su ajuste, resulta indispensable comprender cómo se lleva a cabo su fase de entrenamiento. El éxito de estos modelos no depende únicamente de su estructura interna, sino

de la correcta configuración de los factores que guían su aprendizaje. Por ello, en las siguientes subsecciones se detallan los hiperparámetros, mecanismos de control y conceptos fundamentales que controlan el proceso de entrenamiento.

### 2.3.1. Epochs y tamaño de lote

Durante el proceso de entrenamiento de una red neuronal profunda, el modelo ajusta sus parámetros internos de forma iterativa a medida que procesa la información. El ritmo y el volumen de datos que se procesan en cada actualización de los pesos están regidos por dos hiperparámetros fundamentales de la arquitectura: la época (*epoch*) y el tamaño de lote (*batch size*).

Una época (*epoch*) se define como un ciclo de aprendizaje completo en el que la totalidad del conjunto de datos de entrenamiento pasa a través de la red neuronal, tanto hacia adelante (*forward pass*) para generar las predicciones, como hacia atrás (*backward pass*) para propagar el error. Dado que los algoritmos de optimización basados en el gradiente son iterativos, una sola época rara vez es suficiente para que el modelo minimice el error adecuadamente. Por lo tanto, el entrenamiento exige realizar múltiples épocas. Sin embargo, un número excesivamente alto de épocas puede derivar en un problema de sobreajuste (*overfitting*), donde el modelo comienza a memorizar los datos de entrenamiento perdiendo su capacidad de generalización [14].

Por otro lado, la cantidad de información que componen los conjuntos de datos actuales (especialmente aquellos con imágenes y vídeos) hace que sea computacionalmente inviable, debido a las limitaciones de memoria de las tarjetas gráficas (*GPU*), procesar todos los ejemplos simultáneamente. Para solucionar esta limitación, el conjunto de entrenamiento se divide en fracciones más pequeñas denominadas lotes (*batches* o *minibatches*). El tamaño de lote determina el número de muestras que la red procesa antes de evaluar el error acumulado y actualizar sus pesos [15].

La elección del tamaño de lote tiene un impacto directo en la estabilidad y velocidad del entrenamiento. Un lote grande ofrece una estimación muy precisa del gradiente, pero requiere una enorme cantidad de memoria *VRAM*. Por el contrario, un tamaño de lote pequeño introduce cierto “ruido” matemático en cada actualización. Curiosamente, este ruido suele ser beneficioso, ya que actúa como un mecanismo de regularización que ayuda a la red a escapar de mínimos locales y a encontrar soluciones que generalizan mejor ante datos nuevos [16].

En el contexto del ajuste fino (*fine-tuning*) de arquitecturas masivas, como los *Transformers* o los *LLMs* empleados en este trabajo, el enorme peso de los modelos obligan a utilizar tamaños de lote muy reducidos. Asimismo, dado que estas redes ya cuentan con un amplio conocimiento adquirido durante su preentrenamiento, el número de épocas ne-

cesarias suele ser considerablemente bajo (generalmente entre 3 y 10). Un entrenamiento prolongado en estas condiciones no solo sería ineficiente, sino que podría provocar lo que se conoce como *catastrophic forgetting*, el cual destruiría las representaciones lingüísticas o visuales que el modelo ya había aprendido previamente.

### 2.3.2. Tasa de aprendizaje y *weight decay*

La tasa de aprendizaje (*learning rate*) es, posiblemente, el hiperparámetro más crítico en la configuración de cualquier algoritmo de optimización basado en el gradiente. Este valor determina el tamaño del paso que da el modelo en el espacio de parámetros durante cada iteración para minimizar la función de pérdida. Su correcta elección tiene un impacto significativo en la convergencia del modelo: si la tasa de aprendizaje es demasiado alta, los saltos en la actualización de los pesos serán desproporcionados, lo que puede provocar que el modelo sobrepase el mínimo óptimo y el entrenamiento se vuelva inestable. Por el contrario, si la tasa es excesivamente baja, la red tardará demasiado tiempo en converger o correrá el riesgo de quedarse estancada en un mínimo local [17].

Para complementar el proceso de optimización y mitigar el riesgo de sobreajuste, es práctica común incorporar una técnica de regularización conocida como *weight decay* (decaimiento de pesos). Esta técnica consiste en añadir un término de penalización a la función de pérdida que es proporcional al tamaño de los pesos de la red. De este modo, el algoritmo se ve forzado a mantener los valores de los parámetros lo más pequeños y distribuidos posible. Al evitar que unos pocos pesos adquieran valores desproporcionados, se asume que el modelo resultante es más simple y estable, lo que mejora drásticamente su capacidad para generalizar ante datos no vistos en lugar de memorizar el ruido del conjunto de entrenamiento [18].

En el ámbito del procesamiento del lenguaje natural y la visión por computador con arquitecturas profundas, ambos conceptos suelen trabajar de la mano mediante optimizadores avanzados como *AdamW*. Al realizar el *fine-tuning* de modelos masivos que ya poseen un conocimiento previo (*Transformers* o *LLMs*), la teoría dictamina el uso de tasas de aprendizaje muy reducidas, a menudo acompañadas de planificadores (*schedulers*) que disminuyen este valor dinámicamente con el tiempo. El uso simultáneo de un *learning rate* bajo y un *weight decay* adecuado garantiza que el modelo se adapte a la nueva tarea de clasificación sin destruir las representaciones que adquirió durante su fase de preentrenamiento [19].

### 2.3.3. Función de pérdida

Durante la fase de entrenamiento, el modelo necesita una métrica cuantitativa que evalúe la precisión de sus predicciones frente a las etiquetas reales del conjunto de datos. Esta métrica es la función de pérdida (*loss function*). Su objetivo matemático es calcular el error cometido en cada predicción, devolviendo un valor que el algoritmo de optimización (mediante *backpropagation*) intentará minimizar iterativamente actualizando los pesos de la red [20]. En los problemas de clasificación, las funciones de pérdida basadas en la teoría de la información, concretamente en el concepto de entropía, son el estándar.

Para tareas de clasificación binaria o multiclase, donde las categorías son mutuamente excluyentes (es decir, una entrada solo puede pertenecer a una clase), se emplea la **Entropía Cruzada** (*Cross-Entropy Loss*). Esta función mide la diferencia entre la distribución predicha por el modelo y la distribución real de las etiquetas. Matemáticamente, penaliza de forma logarítmica las predicciones erróneas que el modelo realiza con alta confianza, forzando al sistema no solo a acertar la clase correcta, sino a hacerlo con la mayor certeza posible [21].

Por otro lado, para abordar la tarea de clasificación multietiqueta (donde un mismo registro puede presentar varias categorías de sexismo simultáneamente), el problema se descompone en múltiples clasificaciones independientes. Al tratar cada etiqueta por separado, la entropía cruzada mencionada anteriormente no es aplicable. En su lugar, se utiliza la **Entropía Cruzada Binaria** (*Binary Cross-Entropy* o *BCE*). Esta variante evalúa la probabilidad de presencia o ausencia de cada etiqueta de manera independiente, permitiendo que el modelo determine si una categoría específica aplica o no a la entrada, sin que dicha decisión afecte a las probabilidades del resto de etiquetas [22].

### 2.3.4. Overfitting y técnicas de regularización

Uno de los desafíos más críticos durante el entrenamiento de redes neuronales profundas es el fenómeno conocido como sobreajuste (*overfitting*). Este problema ocurre cuando un modelo es excesivamente complejo en relación con la cantidad de datos de entrenamiento disponibles. En lugar de extraer patrones generales y subyacentes, la red comienza a memorizar el ruido, las anomalías y los detalles específicos del entrenamiento. Como resultado, el modelo alcanza un rendimiento excelente sobre los datos con los que ha sido entrenado, pero fracasa drásticamente al intentar generalizar y predecir sobre datos nuevos o no vistos previamente [23].

Para mitigar el problema y forzar al modelo a aprender representaciones más robustas, se emplean diversas técnicas de regularización. Una de las estrategias más extendidas, que actúa de manera complementaria a la penalización de pesos (*weight decay*) vista en apartados anteriores, es el *Dropout* (abandono).

El *Dropout* es una técnica de regularización estructural que consiste en desactivar aleatoriamente una proporción de las neuronas de la red, junto con sus respectivas conexiones, durante cada iteración (*forward* y *backward pass*) de la fase de entrenamiento. Al “apagar” distintas neuronas en cada paso, se evita que las unidades de la red desarrollen una dependencia mutua excesiva. Esto obliga al modelo a distribuir el aprendizaje, garantizando que la predicción final no dependa de un pequeño grupo de neuronas fuertemente conectadas, sino del conjunto de múltiples sub-redes independientes [24].

En las arquitecturas profundas basadas en mecanismos de atención, como los *Transformers* utilizados en este trabajo tanto para el procesamiento textual como visual, el *Dropout* es un componente fundamental que se encuentra integrado por defecto en el diseño de sus bloques internos. La combinación del *Dropout* arquitectónico con estrategias de optimización regularizadas asegura que el modelo mantenga su capacidad de generalización al adaptarse a la nueva tarea de clasificación.

### 2.3.5. Early stopping y validación

Para monitorizar de forma objetiva el rendimiento de un modelo durante su fase de entrenamiento y detectar a tiempo un posible sobreajuste, resulta imprescindible el uso de un conjunto de validación. Este subconjunto de datos, que se mantiene independiente de las muestras empleadas para actualizar los pesos de la red, permite evaluar iterativamente la capacidad de generalización del modelo. Este proceso se da al finalizar cada época o tras un número determinado de iteraciones y se calcula la función de pérdida y otras métricas de rendimiento sobre este conjunto de validación.

La observación continua de estas métricas es la base del *early stopping* (parada temprana), una de las técnicas de regularización dinámica más eficaces en el aprendizaje profundo. Esta estrategia consiste en interrumpir el entrenamiento de forma automática antes de completar todas las épocas definidas si se detecta que el rendimiento en validación se estanca o empieza a degradarse, mientras que el error de entrenamiento sigue disminuyendo. Este punto de inflexión es el indicador clave de que el modelo ha dejado de aprender patrones generales y ha empezado a memorizar el conjunto de datos de entrenamiento [25].

En la práctica, el *early stopping* se configura mediante un parámetro conocido como paciencia (*patience*), el cual define cuántas evaluaciones sin mejora se permiten antes de detener el proceso. Además de ahorrar valiosos recursos computacionales, esta técnica permite restaurar la configuración de pesos óptima (aquella que obtuvo el mejor resultado en la fase de validación) en lugar de retener los pesos deteriorados de la última época.

Todos estos elementos analizados en las subsecciones anteriores son fundamentales para diseñar modelos robustos y reproducibles. Además, permiten explicar por qué un modelo ha funcionado mejor que otro cuando se ajustan frente a un problema complejo, más

allá de lo que dictan las métricas de evaluación. Como señalan Bergstra y Bengio [26], el conocimiento de estos hiperparámetros y mecanismos de control es clave para mejorar la eficacia y garantizar el éxito de los modelos de aprendizaje automático.

## 2.4. Procesamiento del Lenguaje Natural

El Procesamiento del Lenguaje Natural (PLN) es una rama de la inteligencia artificial y lingüística computacional que se ocupa de la interacción entre los sistemas informáticos y el lenguaje humano. Su objetivo principal es dotar a las máquinas de la capacidad necesaria para comprender, interpretar, generar y analizar el lenguaje natural, ya sea en formato textual o hablado. En el contexto de este Trabajo de Fin de Máster, el PLN constituye un pilar fundamental para extraer el significado y la intención de los vídeos analizados, permitiendo detectar patrones en el discurso asociados a sesgos de género, misoginia y lenguaje sexista.

Históricamente, los sistemas de clasificación textual dependían de un preprocesamiento manual y exhaustivo. Este proceso clásico implicaba la aplicación de técnicas morfológicas para limpiar y normalizar los datos, tales como la división del texto en unidades mínimas (*tokenización* por palabras), la eliminación de términos que no tienen una carga semántica dentro de la frase (*stopwords*), la lematización y el truncamiento (*stemming*). Tras esta fase, el texto se transformaba en matrices numéricas basadas en la frecuencia de aparición de los términos (como *TF-IDF* o *Bag-of-Words*). Sin embargo, estos enfoques tradicionales presentaban una limitación estructural crítica: al ignorar el orden de las palabras, se perdía gran parte del contexto y de la semántica subyacente de las frases [27].

En la actualidad, la disciplina del PLN ha experimentado una profunda evolución gracias a la adopción de nuevas arquitecturas profundas de aprendizaje automático. Los nuevos modelos, especialmente aquellos basados en la arquitectura *Transformer*, han transformado el paradigma del análisis textual. En lugar de requerir una limpieza léxica manual, estos modelos utilizan tokenizadores de subpalabras (como *Byte-Pair-Encoding*) y proyectan el texto en espacios vectoriales continuos de alta dimensión denominados *embeddings*. Esta representación vectorial permite al modelo conservar de manera simultánea tanto la información sintáctica como la riqueza semántica. Al procesar secuencias de forma bidireccional y contextual, el sistema es capaz de captar matices lingüísticos complejos, como la ironía, el sarcasmo o el doble sentido, elementos que son intrínsecos al lenguaje natural y vitales para detectar el discurso sexista [28].

### 2.4.1. Preprocesamiento del texto

Para que un modelo de aprendizaje profundo pueda procesar y extraer valor de la información textual, los datos deben someterse a una fase previa de acondicionamiento. Tal y como se ha mencionado en la sección anterior, aunque las arquitecturas basadas en *Transformers* no requieren técnicas clásicas de limpieza lingüística (como la lematización), sí exigen un proceso de eliminación de ruido específico del dominio y una tokenización matemática estricta.

En la fase de experimentación de este trabajo, las transcripciones y textos analizados se han sometido a una tubería de limpieza (*pipeline*) mediante el uso de expresiones regulares. El objetivo ha sido estandarizar el vocabulario (convirtiendo todo el texto a minúsculas) y eliminar aquellos elementos sin valor semántico que introducen ruido en la red. Concretamente, se ha procedido a la eliminación de enlaces web (*URLs*), menciones a otros usuarios, etiquetas de metadatos (*hashtags*), emoticonos (*emojis*) y etiquetas estructurales derivadas de la transcripción de los vídeos (como los prefijos “AUDIO:” o “VIDEO:”). La supresión de estos elementos resulta fundamental para aislar el discurso real y evitar que el modelo dedique recursos a aprender patrones sintácticos irrelevantes para la tarea de clasificación [29].

Una vez que el texto ha sido limpiado, se aplica el proceso de tokenización. Las nuevas arquitecturas emplean algoritmos de subpalabras, como *Byte-Pair Encoding* (BPE) o *WordPiece*. Estos enfoques son altamente eficientes, ya que conservan las palabras comunes enteras, pero dividen los términos raros, mal escritos o específicos de la jerga de internet en subunidades más pequeñas y conocidas.

Finalmente, el tokenizador actúa como puente entre el texto y la red neuronal: toma estas subpalabras y las convierte en vectores de identificadores numéricos (*input IDs*). Para garantizar que todas las secuencias adquieran una dimensión matricial uniforme antes de ser procesadas por el modelo, se aplica un proceso de truncamiento (estableciendo una longitud máxima permitida para los registros demasiado largos) y de relleno (*padding*) para los textos más cortos. Adicionalmente, el tokenizador genera estructuras de control, como las máscaras de atención (*attention masks*), las cuales indican al algoritmo qué partes del vector contienen información real y cuáles son simples rellenos. Este formato numérico es el único lenguaje que los modelos de aprendizaje profundo pueden procesar internamente.

### 2.4.2. Representación del texto

Una vez que el texto ha sido tokenizado, el siguiente reto del PLN es transformar esos identificadores numéricos en un formato que capture el significado lingüístico. Históricamente, esta tarea se realizaba mediante técnicas de extracción de características basadas en frecuencias, como el *Bag-of-Words* o *TF-IDF*. Aunque resultan útiles en algoritmos



de aprendizaje automático clásico, estas estrategias generan representaciones dispersas y asumen la independencia entre los términos, fallando a la hora de capturar las relaciones semánticas intrínsecas.

Para superar estas limitaciones, se introdujeron los *Word Embeddings* o representaciones vectoriales densas. En términos generales, un *Word Embedding* proyecta cada palabra como un vector de números reales en un espacio continuo de alta dimensión, lo que permite cuantificar el contexto de una palabra, así como su similitud semántica y sintáctica, mediante distancias matemáticas.

Uno de los enfoques pioneros en este ámbito fue *Word2Vec*, el cual utiliza redes neuronales poco profundas para aprender representaciones vectoriales mediante dos posibles arquitecturas: *Continuous Bag-of-Words (CBOW)*, que predice una palabra objetivo a partir de su contexto, o *Skip-Gram*, que realiza la tarea inversa prediciendo el contexto a partir de una palabra principal. Posteriormente, surgieron alternativas como *GloVe (Global Vectors for Word Representation)*, un modelo que combina las ventajas de los métodos de ventana de contexto local con la factorización matricial global del corpus.

Sin embargo, estos modelos estáticos presentaban una limitación estructural sin solución: el problema de la polisemia. Dado que algoritmos como *Word2Vec* o *GloVe* precálculan un único vector inamovible para cada término en el vocabulario, son incapaces de distinguir cuándo una misma palabra tiene significados distintos en función del contexto en la oración [30].

Esta debilidad impulsó el desarrollo de la actual generación de representaciones contextuales densas. Las nuevas arquitecturas, en lugar de consultar un diccionario estático de vectores, calculan la representación numérica de cada subpalabra de manera dinámica, analizando simultáneamente toda la secuencia que la rodea. Esta necesidad de contextualización profunda dio lugar a las arquitecturas basadas en mecanismos de atención, las cuales se abordarán en detalle en las siguientes secciones.

### 2.4.3. Clasificación de texto en PLN

La clasificación de textos es una de las tareas computacionales más fundamentales dentro del PLN. Su objetivo principal consiste en asignar, de forma automática, una o varias categorías semánticas predefinidas a un fragmento de texto no estructurado. Dependiendo de la complejidad y la naturaleza del problema, los sistemas de clasificación se dividen en tres grandes grupos: clasificación binaria, multiclase y multietiqueta, las cuales ya fueron definidas anteriormente en el apartado 2.2 de esta memoria.

En el contexto de este Trabajo de Fin de Máster, la clasificación de texto se emplea como motor analítico para auditar las transcripciones de los videos seleccionados. El proceso se aborda desde una doble vertiente. En primer lugar, se aplica una clasificación binaria para

dictaminar si el contenido expresa algún tipo de lenguaje sexista o misógino. En caso afirmativo, el problema evoluciona hacia un esquema de clasificación más fino y detallado, cuyo objetivo es identificar qué tipos específicos de sesgos, roles de género o violencias discursivas están presentes en el mensaje.

La relevancia de este desafío tecnológico y social queda patente en las prestigiosas campañas de evaluación competitiva organizadas por congresos internacionales como *IberLEF* o *CLEF*. De hecho, el marco experimental y metodológico desarrollado en este trabajo se fundamenta directamente en la competición *EXIST 2025 (sEXism Identification in Social neTworks)*.

Concretamente, los modelos implementados en este proyecto dan respuesta a los retos planteados en la **Tarea 3.1** de dicha competición, centrada en la detección binaria de misoginia en redes sociales, y en la **Tarea 3.3**, enfocada en la categorización avanzada y el análisis multietiqueta del discurso machista. La participación y adaptación a este tipo de iniciativas garantiza que las soluciones aquí propuestas operan sobre un conjunto de datos estandarizado y contribuyen activamente al estado del arte en la mitigación del discurso de odio en entornos digitales.

## 2.5. Modelos Avanzados para Texto

La necesidad de capturar el contenido dinámico y resolver el problema de la polisemia en la representación textual ha propiciado el desarrollo de arquitecturas neuronales de última generación. En esta sección se describen los fundamentos de los modelos basados en mecanismos de atención y su evolución hasta los modernos sistema generativos, los cuales constituyen el núcleo de las tecnologías aplicadas en este trabajo.

### 2.5.1. Transformers

La arquitectura *Transformer*, la cual se puede apreciar en la [Figura 2.3](#), supuso un cambio de paradigma en el PLN tras ser propuesta en el influyente artículo *Attention is all you need* [31]. A diferencia de los modelos recurrentes tradicionales, como las Redes Neuronales Recurrentes (RNN) o las redes de memoria a corto-largo plazo (*LSTM*), los *Transformers* prescinden del procesamiento secuencial. Esta característica permite analizar las secuencias de texto en paralelo, lo que acelera drásticamente los tiempos de entrenamiento y facilita la captura de dependencias a muy largo plazo.

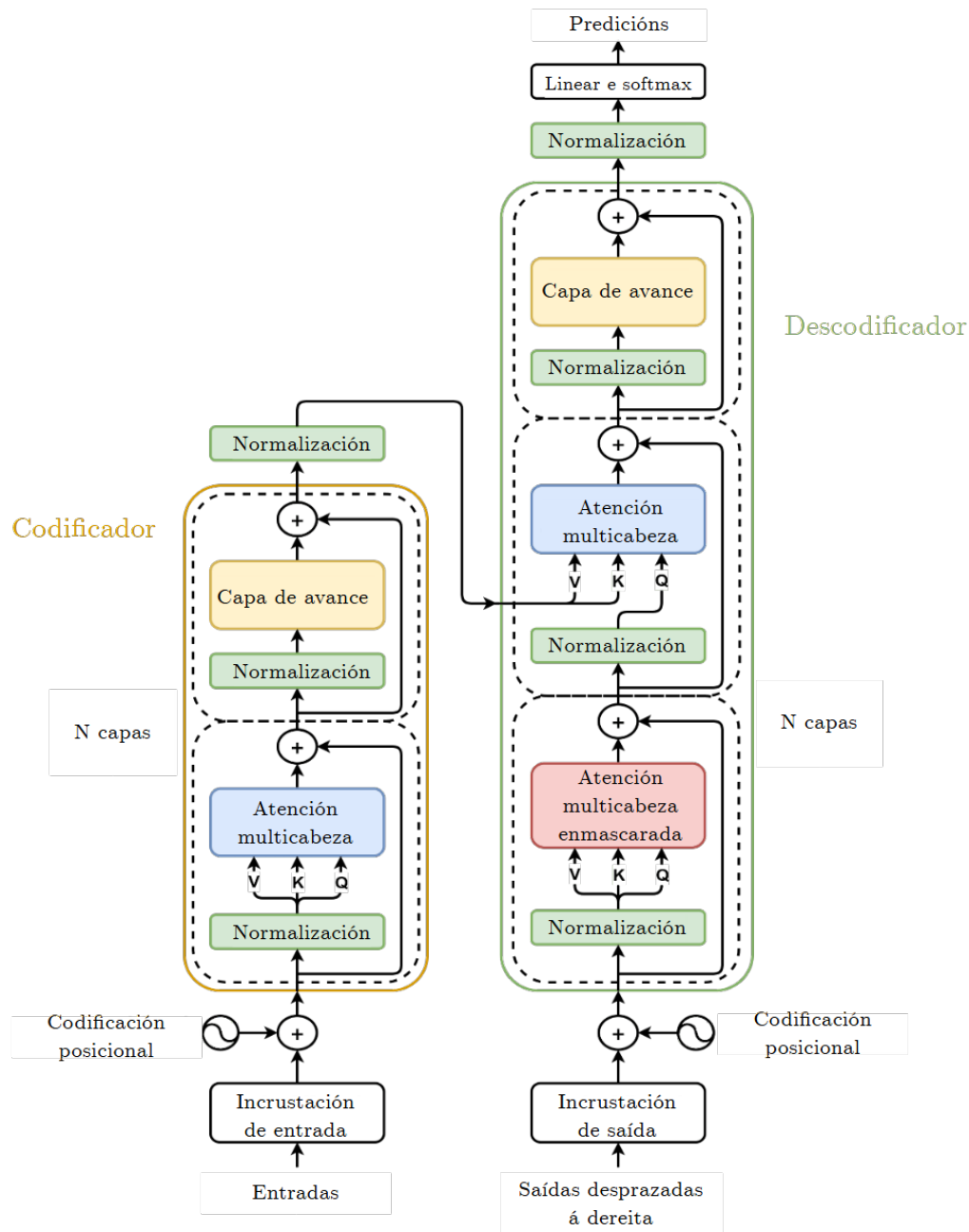


Figura 2.3: Arquitectura de los modelos *Transformers*

El pilar de esta arquitectura es el mecanismo de autoatención (*self-attention*), cuyo funcionamiento se ejemplifica gráficamente en la [Figura 2.4](#). Este mecanismo relaciona diferentes posiciones de una simple secuencia para computar una representación global de la misma. Matemáticamente, asigna una consulta (*query*) y un conjunto de pares clave-valor (*key-value*) a una salida, calculando pesos que determinan la importancia relativa de cada palabra respecto a las demás dentro de la misma oración.

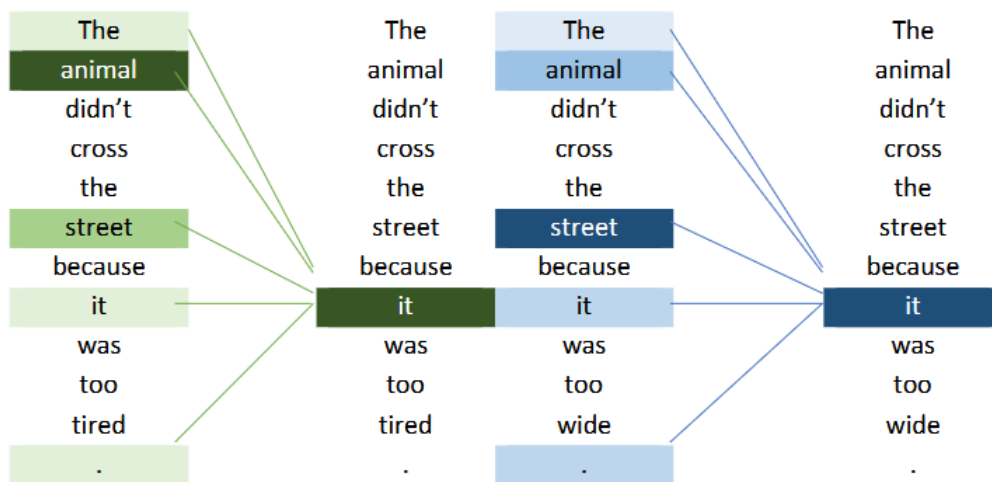
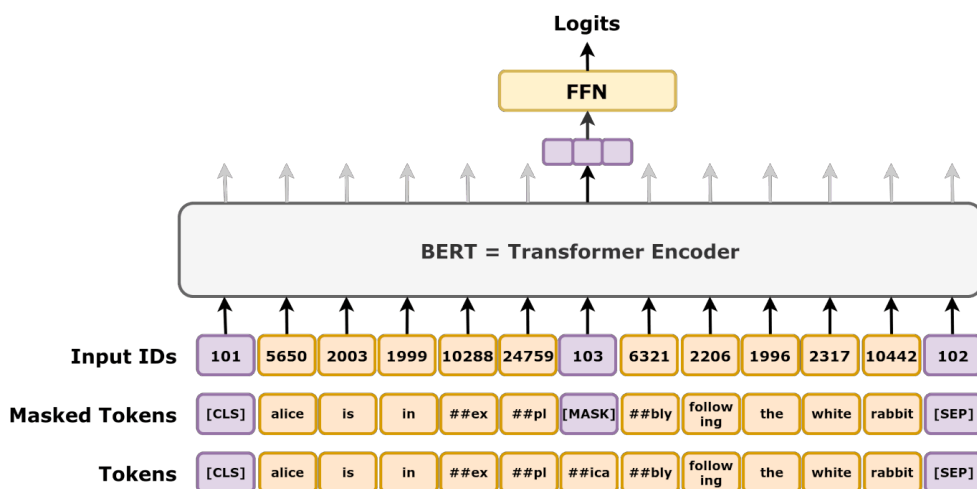


Figura 2.4: Ejemplo de funcionamiento del mecanismo de atención

Esta innovación dio lugar a una nueva generación de modelos de lenguaje preentrenados fundamentados en la codificación bidireccional, siendo *BERT* (*Bidirectional Encoder Representations from Transformers*) [32] el exponente clásico. Este necesita de las entradas que se muestran en la Figura 2.5. *BERT* es capaz de aprender el contexto de una palabra observando su secuencia antecesora y predecesora de manera simultánea. Posteriormente, surgieron optimizaciones robustas como *RoBERTa* (*Robustly Optimized BERT Approach*) [33], que mejora el rendimiento eliminando la predicción de la siguiente oración durante el preentrenamiento, utilizando secuencias más largas e introduciendo un enmascaramiento dinámico de los datos.

Figura 2.5: Entradas necesarias para el codificador de *BERT*

### Modelos implementados en este trabajo

Para abordar la complejidad de esta tarea y asegurar una evaluación exhaustiva, en el desarrollo empírico de este Trabajo de Fin de Máster se ha implementado una amplia batería de arquitecturas discriminativas basadas en *Transformers*, adaptadas a diferentes dominios lingüísticos:

- **Modelos Multilingües:** Se han utilizado `mmBERT`, `xlm-roberta-base` y `TwHIN-BERT`, idóneos para captar estructuras semánticas universales y transferir conocimiento entre idiomas.
- **Modelos específicos para Español:** Se han evaluado `BETO`, `BERTin` y `RoBERTuito`, siendo este último optimizado para el lenguaje informal de redes sociales.
- **Modelos específicos para Inglés:** Se ha experimentado con `RoBERTa-Base`, `BERTweet` (preentrenado en mensajes cortos y jerga digital) y `Electra-base-discriminator` (que introduce un paradigma de reemplazo de tokens más eficiente computacionalmente).

#### 2.5.2. Modelos Generativos de Lenguaje (LLMs)

Paralelamente a la consolidación de modelos discriminativos como *BERT*, la evolución del aprendizaje profundo ha propiciado la explosión de los Modelos Generativos de Lenguaje (*Large Language Models*) [34]. A diferencia de los enfoques anteriores, los cuales están entrenados para comprender y clasificar, los *LLMs* utilizan arquitecturas autorregresivas fundamentadas habitualmente en la parte decodificadora del *Transformer*. Su objetivo matemático principal es predecir el siguiente token de una secuencia para generar texto altamente coherente y contextualizado.

Modelos como `GPT-3`, `GPT-4`, `LLaMA` o `PaLM` se entrenan sobre volúmenes masivos de datos no estructurados. Gracias a este vasto conocimiento subyacente, han demostrado capacidades de generalización sorprendentes. Una de las ventajas más potentes de los *LLMs* es su capacidad para ejecutar tareas supervisadas a través del diseño de instrucciones o *prompting*. Existen tres estrategias principales de aplicación:

- **Zero-shot:** El modelo realiza la tarea directamente con una única instrucción, sin recibir ejemplos previos. Es idóneo cuando se busca rapidez o no se dispone de datos etiquetados.
  - *Ejemplo:* “¿La siguiente transcripción de un vídeo de TikTok a texto contiene lenguaje sexista? Texto: ‘Las mujeres no sirven para liderar proyectos.’ Responde únicamente Sí o No”
- **Few-shot:** Se incluyen varios ejemplos demostrativos dentro de la propia instrucción

para que el modelo identifique el patrón deseado antes de emitir su respuesta.

- *Ejemplo:*
  - Texto de ejemplo: “Todos merecemos las mismas oportunidades.” → No sexista.
  - Texto de ejemplo : “Las chicas son demasiado emocionales para los negocios.” → Sexista.
  - Texto nuevo: “Ellas deberían quedarse cuidando la casa.” → El modelo infiere y dictamina que es Sexista.
- **Fine-tuning (Ajuste fino):** Consiste en entrenar los pesos del modelo preentrenado utilizando un conjunto de datos específicos del dominio. Aunque requiere mayores recursos computacionales, suele ofrecer un rendimiento superior y constante.

### Aplicación práctica en este trabajo: *PEFT* y *QLoRA*

En el marco experimental de este proyecto se ha apostado firmemente por el ajuste fino. Sin embargo, debido al masivo número de parámetros de los *LLMs* modernos, el reentrenamiento completo resulta computacionalmente inviable. Para solucionar este obstáculo, se ha implementado una técnica **QLoRA** (*Quantized Low-Rank Adaptation*) [35] sobre tres modelos generativos de última generación: *Mistral-7B-Instruct-v0.3*, *meta-llama/Llama-3.1-8B-Instruct* y *google/gemma-2-2b-it*. Esta metodología ha permitido cuantizar los pesos base de los modelos a una precisión de 4 bits, congelarlos, y entrenar únicamente matrices de bajo rango insertadas en las proyecciones matemáticas de atención.

Para adaptar estos modelos a la clasificación binaria de las transcripciones de vídeo, se han desarrollado dos estrategias arquitectónicas distintas:

- **Enfoque Discriminativo (*Sequence Classification*):** En los modelos *Mistral* y *Gemma*, se sustituyó la cabeza de modelado de lenguaje original por una capa lineal de clasificación pura (`AutoModelForSequenceClassification`), ajustando los pesos de esta nueva capa (`score`) para extraer directamente la probabilidad matemática de que el texto contenga lenguaje misógino.
- **Enfoque Generativo Autorregresivo (*Causal LM*):** En el modelo de la familia *LLaMa*, se mantuvo la arquitectura generativa intacta (`AutoModelForCausalLM`). El ajuste fino se realizó formateando las instrucciones de entrada (*prompts*) para forzar al modelo a predecir y autocompletar exclusivamente un token de clasificación (“YES” para contenido sexista o “NO” para contenido no sexista) mediante un optimizador supervisado (`SFTTrainer`).

### Ventajas y limitaciones de la clasificación con LLMs

La adopción de estas arquitecturas masivas para tareas de clasificación del discurso de odio es una tendencia en alza, aunque presenta características contrastadas:

- **Ventajas:** Permiten aprovechar un conocimiento semántico universal y el contexto general del mundo, se adaptan rápidamente a nuevas definiciones de categorías, y alcanzan un nivel de comprensión profundo de la ironía o el sarcasmo.
- **Limitaciones:** Su naturaleza generativa los hace susceptibles a sufrir alucinaciones o generar salidas inconsistentes si no se ajustan correctamente (como ocurre con la modalidad *zero-shot*), exhiben una alta sensibilidad al diseño de la instrucción (*prompt*) y conllevan un coste computacional elevado para inferencias masivas.

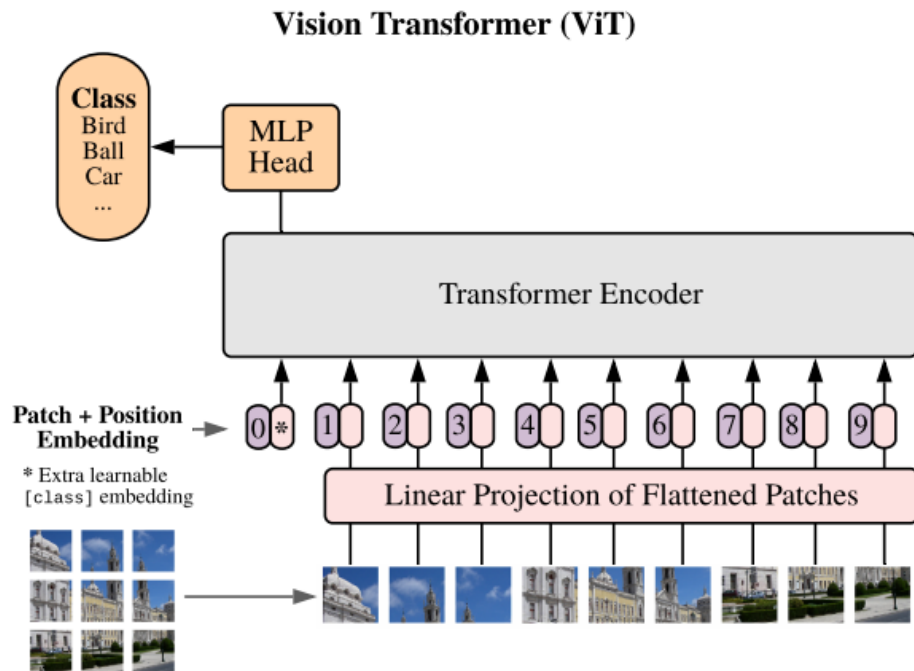
## 2.6. Modelos Avanzados para Visión Estática (Imágenes)

De manera análoga a la evolución experimentada en el PLN, la visión por computador ha sufrido una profunda transformación en los últimos años. Una de las estrategias más eficientes para el análisis de vídeo consiste en eludir el altísimo coste computacional de las arquitecturas tridimensionales (*3D-CNNs*) [36] extrayendo secuencias de fotogramas clave y tratándolos como imágenes estáticas independientes. Para procesar estas imágenes, el estado del arte actual se divide en dos grandes enfoques según su topología.

### 2.6.1. Vision Transformers (ViT) y variantes jerárquicas

Inspirados por el abrumador éxito de los mecanismos de autoatención en el texto conseguido gracias a los *Transformers*, en 2020 se introdujeron los *Vision Transformers* (*ViT*) [37], demostrando que la dependencia de las redes convolucionales tradicionales no era estrictamente necesaria para tareas de visión complejas.

La arquitectura *ViT* divide la imagen original en una cuadrícula de pequeños parches (por ejemplo, 16x16 píxeles). Tal y como se muestra en la [Figura 2.6](#) cada parche es aplanado y proyectado linealmente para formar un vector, operando funcionalmente como si fuera un *token* o palabra en una frase. Tras añadir *embeddings* posicionales para retener la información espacial, esta secuencia de parches se introduce directamente en el codificador de un *Transformer* estándar. Al procesar las relaciones globales entre todos los parches mediante la autoatención, *ViT* es capaz de captar un contexto visual integral desde las primeras capas.

Figura 2.6: Arquitectura de un *ViT*

A pesar de su éxito, *ViT* presenta una complejidad computacional cuadrática respecto a la resolución de la imagen. Para solventar este problema, surgieron variantes como *Swin Transformer (Shifted Window Transformer)* [38]. Esta arquitectura jerárquica computa la atención de manera local dentro de ventanas no superpuestas que se van desplazando entre capas, lo que otorga mayor flexibilidad y eficiencia a la hora de procesar distintas escalas visuales.

### 2.6.2. Redes Neuronales Convolucionales Modernas (*ConvNeXt*)

El auge de los *ViT* llevó a repensar y modernizar el diseño clásico de las Redes Neuronales Convolucionales (*CNNs*). Investigadores de *Facebook* y de la Universidad de California propusieron *ConvNeXt* [39], una arquitectura puramente convolucional construida a partir de una red residual estándar (*ResNet*) [40], pero incorporando las decisiones de diseño que hicieron triunfar a los *Transformers*.

Entre estas mejoras estructurales destacan el uso de tamaños de filtro (*kernel*) más grandes, la sustitución de la normalización por lotes (*Batch Normalization*) por la normalización de capas (*Layer Normalization*), y el uso de funciones de activación más suaves. El resultado es una red que sostiene la propia eficiencia computacional de las *CNNs*, pero que logra igualar o incluso superar la precisión de arquitecturas basadas en mecanismos de



atención como *Swin Transformer*.

### 2.6.3. Agregación temporal para clasificación de vídeo

Cuando estas arquitecturas de imagen estática se aplican a la clasificación de vídeos (como es el caso de este trabajo), el sistema emite una predicción matemática probabilística de forma aislada para cada fotograma extraído de la secuencia temporal. Dado que el objetivo final es emitir un único veredicto global para todo el archivo multimedia, resulta teóricamente indispensable aplicar heurísticas de agregación o *pooling*.

En la literatura científica, las estrategias de consolidación más habituales para agrupar estas predicciones independientes son:

- **Max-Pooling (sensibilidad máxima)**: Consiste en evaluar el valor máximo de probabilidad de toda la secuencia. Entonces, si un solo fotograma contiene evidencias claras que superan el umbral de activación, todo el vídeo hereda dicha clasificación. Es ideal para detectar anomalías o infracciones breves pero críticas.
- **Average-Pooling (Promedio continuo)**: Se calcula la media aritmética de las probabilidades de todos los fotogramas. Esta técnica exige que el contenido mantenga un nivel de confianza sostenido a lo largo del tiempo para influir en el veredicto final.
- **Majority Vote (votación por mayoría)**: Cada fotograma emite un voto discreto (positivo o negativo). El vídeo adopta la clasificación que haya obtenido si supera un umbral definido previamente (lo que se consideraría como mayoría). Es una estrategia altamente robusta frente a fotogramas ruidosos, corruptos o atípicos (*outliers*).

Los detalles específicos sobre la selección de estos modelos, el volumen de fotogramas extraídos por vídeo y la implementación práctica de estas métricas de agregación se detallarán exhaustivamente en el capítulo correspondiente a la metodología experimental.

## 2.7. Modelos Avanzados para Visión Temporal (Vídeos)

Aunque el análisis basado en la extracción de fotogramas estáticos reduce drásticamente la carga computacional, esta metodología descarta una dimensión fundamental de los archivos multimedia: la progresión temporal. Para comprender acciones complejas, ironías o comportamientos que se desarrollan a lo largo de un clip, los sistemas de inteligencia artificial deben ser capaces de modelar conjuntamente el espacio (los píxeles en un instante dado) y el tiempo (la evolución de dichos píxeles a través de los fotogramas). En esta sección se abordan las arquitecturas modernas diseñadas para procesar el vídeo como un volumen continuo.

### 2.7.1. Adaptación de la Arquitectura *Transformer* al Vídeo

La aplicación directa del mecanismo de *self-attention* a todos los píxeles de todos los fotogramas de un vídeo resulta computacionalmente inviable, ya que la complejidad matemática crece de forma cuadrática respecto a la longitud de la secuencia de entrada. Para solucionar este enorme cuello de botella, la literatura científica ha propuesto adaptaciones eficientes que extienden el éxito del *ViT* al ámbito tridimensional.

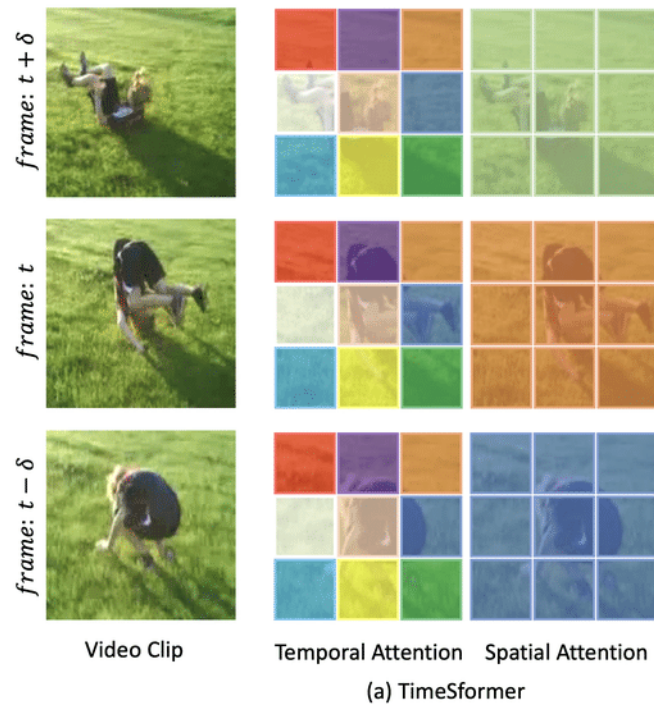


Figura 2.7: Ilustración de los mecanismos de atención de un *TimeSformer*

Dentro de este campo, destacan dos enfoques arquitectónicos fundamentales implementados en el desarrollo de este trabajo:

- ***TimeSformer (Time-Space Transformer)***: Propuesto por investigadores de *Facebook* [41], este modelo resuelve el problema de la explosión paramétrica mediante la **atención dividida**. En lugar de comparar cada parche con todos los demás parches del vídeo simultáneamente, *TimeSformer* realiza el cálculo en dos pasos: primero aplica una atención puramente espacial (comparando los parches dentro de un mismo fotograma) y, a continuación, aplica una atención puramente temporal (comparando el mismo parche exclusivamente con sus homólogos en los fotogramas anteriores y posteriores), tal y como se puede observar en la [Figura 2.7](#).
- ***ViViT (Video Vision Transformer)***: Desarrollado por investigadores de *Google* [42], es la evolución directa del *ViT* estático. Su mayor innovación es el cambio en la forma de extraer las características iniciales: en lugar de dividir la imagen en parches bidimensionales (2D), *ViViT* extrae lo que se conoce como ***Tubelets*** o

tubos tridimensionales. Como se puede apreciar en la [Figura 2.8](#), estos tubos agrupan un bloque de píxeles que atraviesa varios fotogramas consecutivos, permitiendo a la red capturar dinámicas espaciotemporales (como la dirección y la velocidad de un movimiento) desde la primera capa del codificador.

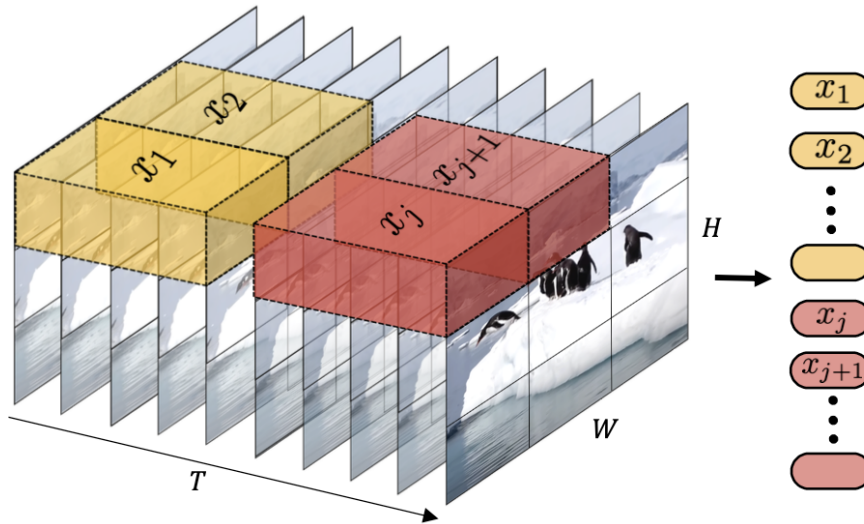


Figura 2.8: Representación de los *Tubelets* de los *ViViT*

### 2.7.2. Enfoques Basados en Autoencoders Enmascarados

Paralelamente a las adaptaciones topológicas del *Transformer*, el entrenamiento de modelos de vídeo ha experimentado una gran revolución gracias a las técnicas de aprendizaje autosupervisado. El principal reto del aprendizaje en vídeo es la **redundancia de la información**: dos fotogramas consecutivos suelen ser casi idénticos, lo que provoca que las redes neuronales colapsen o memoricen atajos visuales en lugar de aprender características profundas.

Para combatir este fenómeno, surgieron modelos como ***VideoMAE*** (*Video Masked Autoencoders*) [43], una arquitectura inspirada en los métodos de entrenamiento con enmascaramiento nacidos en el PLN (como *BERT*).

El funcionamiento de *VideoMAE* se basa en un paradigma de reconstrucción extrema. Tal y como podemos observar en la [Figura 2.9](#), el modelo toma un volumen de vídeo (dividido en cubos espaciotemporales) y **oculta deliberadamente hasta el 90 % de la información**. El codificador (*encoder*) solo procesa el 10 % de los parches visibles, y posteriormente, un decodificador (*decoder*) es forzado a reconstruir todos los píxeles originales que faltan. Este nivel de enmascaramiento tan agresivo impide que la red se apoye en la redundancia temporal de los píxeles vecinos, obligándola a desarrollar una comprensión semántica de muy alto nivel sobre la física, el movimiento y la estructura de

los objetos que aparecen en pantalla.

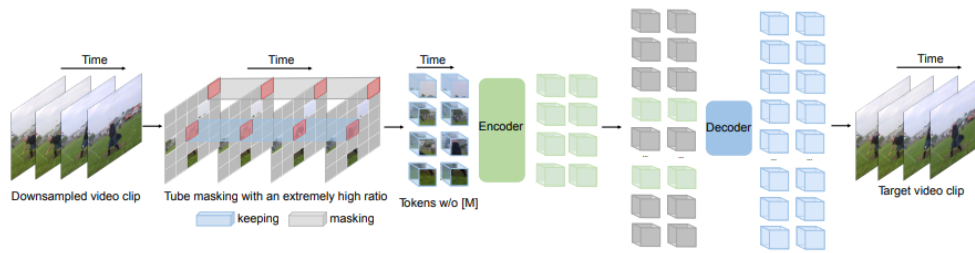


Figura 2.9: Arquitectura de *VideoMAE*

## 2.8. Aprendizaje por Transferencia

El aprendizaje por transferencia (*Transfer Learning*) ha revolucionado el desarrollo de la inteligencia artificial, consolidándose como el enfoque estándar en el PLN, la Visión Artificial y el aprendizaje multimodal [44]. Este paradigma consiste en reutilizar el conocimiento previamente adquirido por un modelo computacional al resolver una tarea genérica (generalmente entrenado con volúmenes de datos masivos) para aplicarlo a la resolución de una tarea nueva y específica, la cual suele disponer de una cantidad mucho menor de datos etiquetados.

Históricamente, el desarrollo de sistemas de aprendizaje profundo exigía inicializar los pesos de las redes neuronales de forma aleatoria y entrenarlas desde cero, un proceso que requería un enorme coste computacional y bases de datos gigantescas para evitar el sobreajuste. El aprendizaje por transferencia elimina esta barrera: en lugar de aprender a extraer características desde la nada, se parte de un modelo preentrenado que ya posee representaciones universales del lenguaje (como es el caso de *BERT*, *RoBERTa* o *GPT*) o de las imágenes [45]. Este conocimiento base se transfiere y se refina para la nueva tarea, permitiendo democratizar el uso de modelos masivos y obtener resultados competitivos con tiempos de entrenamiento significativamente menores.

### 2.8.1. Fine-tuning y optimización de hiperparámetros

Dentro del aprendizaje por transferencia, la estrategia de adaptación más utilizada y efectiva es el ajuste fino o *fine-tuning*. Este proceso, popularizado en el PLN por enfoques como ULMFiT [46], consiste en tomar la arquitectura de un modelo preentrenado, añadirle una nueva capa de salida orientada a la tarea específica a resolver (por ejemplo, una capa de clasificación binaria) y ajustar los pesos internos de toda la red, o de una parte de ella, utilizando un conjunto de datos propio del dominio.

Sin embargo, la literatura científica ha demostrado que modelos masivos como *BERT* o *RoBERTa* son altamente inestables y sensibles durante esta fase de adaptación [47]. Para que el *fine-tuning* sea exitoso y el modelo no “olvide” catastróficamente lo que aprendió en su preentrenamiento, resulta imprescindible realizar un proceso de **Optimización de Hiperparámetros (HPO)**. A nivel teórico, existen diversas estrategias para explorar el espacio de hiperparámetros: desde la búsqueda exhaustiva (*Grid Search*) y la búsqueda aleatoria (*Random Search*), hasta métodos avanzados como la optimización bayesiana, que modela probabilísticamente el rendimiento de combinaciones pasadas para guiar la búsqueda de forma inteligente [48].

Durante este proceso de optimización, las variables de configuración que rigen cómo aprende el algoritmo son determinantes y se dividen en:

- Por un lado, parámetros estructurales como el *Learning Rate*, *Weight Decay* y *Epochs*, que ya fueron definidos en la [Subsección 2.3.2](#) y [Subsección 2.3.1](#) respectivamente.
- Por otro lado tenemos:
  - **Calentamiento (*Warmup*)**: Consiste en iniciar el entrenamiento con una tasa de aprendizaje cercana a cero y aumentarla gradualmente durante un porcentaje inicial de los pasos, evitando actualizaciones bruscas que desestabilicen el modelo.
  - **Planificador de la tasa (*LR Scheduler*)**: Algoritmo que ajusta de forma dinámica la tasa de aprendizaje a medida que avanzan las épocas (por ejemplo, reduciéndola de forma lineal o mediante una curva coseno).

### Estrategias avanzadas de optimización

En la práctica empírica de entrenamiento de modelos masivos como los de visión estática y los de visión temporal, la combinación de estos hiperparámetros ha dado lugar a estrategias de ajuste específicas:

- **Estrategia conservadora o *Slow Cooker***: Consiste en someter al modelo a un entrenamiento prolongado (mayor número de épocas) utilizando un *learning rate* muy bajo y un *weight decay* excepcionalmente alto (regularización agresiva). Se acompaña de un periodo de *warmup* prolongado. Es ideal para que el modelo aprenda patrones complejos y sutilezas en conjuntos de datos difíciles sin memorizarlos.
- **Simulación de lotes masivos (*Gradient Accumulation*)**: La memoria de las tarjetas gráficas (*VRAM*) suele limitar el tamaño del lote de datos (*batch size*) que se puede procesar simultáneamente. Para lograr gradientes matemáticamente estables como los que ofrecería un lote masivo, se acumulan los gradientes de varios lotes pequeños durante múltiples iteraciones antes de ejecutar la actualización de los pesos.

- **Ajuste agresivo o *Sprint Final*:** Se utiliza una combinación de *learning rate* y regularización moderadas, pero aplicando un planificador tipo coseno (*Cosine Scheduler*). Tras un calentamiento inicial, la tasa de aprendizaje se mantiene estable para luego decaer rápida y suavemente hacia el final, exprimiendo el máximo rendimiento de la red en las últimas iteraciones.

## 2.9. Ensemble de Modelos y Fusión multimodal

En el ámbito del aprendizaje automático, la técnica de *ensemble* (conjunto de modelos) consiste en combinar las predicciones de múltiples algoritmos diversos para generar una única decisión final más robusta, estable y precisa [49]. La premisa matemática detrás de esta técnica es que, al agregar diferentes perspectivas, los errores individuales de cada modelo se compensan mutuamente, reduciendo la varianza y mitigando los sesgos específicos de cada arquitectura.

En el contexto del análisis del discurso de odio en redes sociales, la información es inherentemente multimodal. Un vídeo no se compone de una única fuente de información, sino que abarca la transcripción textual del audio, la composición visual de sus fotogramas estáticos y la evolución temporal de sus movimientos.

### 2.9.1. Ventajas de los *Transformers* en tareas multimodales

La capacidad de los *Transformers* y de las modernas arquitecturas convolucionales para manejar secuencias de cualquier tipo y capturar dependencias a largo plazo los convierte en herramientas extremadamente potentes para tareas multimodales. En lugar de depender de un único modelo monolítico que intente comprender todo a la vez, la tendencia actual se orienta a utilizar arquitecturas expertas dedicadas que procesan el texto, la imagen y el vídeo en paralelo, combinando su información a posteriori para realizar predicciones conjuntas. Este enfoque permite capturar con una altísima precisión las relaciones y contradicciones entre lo que se dice en la transcripción de un vídeo y lo que se muestra en su componente visual.

### 2.9.2. Fusión Temprana vs Fusión Tardía

A nivel teórico, la integración de múltiples modalidades de datos se divide principalmente en dos tipos [50]:

- **Fusión Temprana (*Early Fusion*):** Los datos o características extraídas de diferentes modalidades se combinan en las primeras etapas de la red neuronal, creando un

vector unificado que es procesado por un único clasificador final. Aunque permite que el modelo aprenda interacciones cruzadas desde el principio, es computacionalmente muy costoso y propenso al *overfitting* si una modalidad domina sobre las demás.

- **Fusión Tardía (*Late Fusion*)**: Cada modalidad es procesada por un modelo experto independiente hasta generar una predicción final (una probabilidad matemática). Posteriormente, estas probabilidades aisladas se combinan mediante algoritmos estadísticos para emitir el veredicto definitivo.

En la práctica empírica y en las competiciones de Ciencia de Datos, el *late fusion* mediante predicción estática se ha estandarizado como la metodología más eficiente. Al separar la fase de inferencia de la fase de combinación, esta técnica permite ajustar y recalcular el peso de cada modelos miles de veces en cuestión de milisegundos, sin necesidad de reprocesar repetidamente la costosa extracción de características a través de las tarjetas gráficas (*VRAM*).

### Promedio Ponderado (*Weighted Average*)

Una de las estrategias matemáticas más utilizadas en la fusión tardía es el promedio ponderado. En este método, a la probabilidad emitida por cada modelo experto se le asigna un peso específico ( $w$ ) que refleja su nivel de fiabilidad o importancia empírica en el conjunto del sistema. Matemáticamente, la probabilidad final unificada se calcula del siguiente modo:

$$P_{final} = (w_{texto} \cdot P_{texto}) + (w_{imagen} \cdot P_{imagen}) + (w_{video} \cdot P_{video}) \quad (2.1)$$

Sujeto a la restricción de que la suma de todos los pesos sea exactamente 1. Los valores óptimos de estos pesos no se definen arbitrariamente, sino que se descubren mediante algoritmos de búsqueda y validación cruzada que evalúan qué combinación exacta maximiza las métricas de rendimiento sobre un conjunto de datos estático.

## 2.10. Subjetividad en la anotación de los datos

En los sistemas de aprendizaje automático tradicionales, se asume que existe una única “verdad absoluta” para cada conjunto de datos. Sin embargo, en tareas vinculadas al PLN que involucren un alto componente subjetivo, como la detección del discurso de odio, el sexismo o la toxicidad, esta premisa se desvanece. La percepción de un mensaje como ofensivo está intrínsecamente ligada a la sensibilidad, el contexto cultural y las experiencias vitales de la persona que lo evalúa. Por tanto, el desacuerdo entre los anotadores humanos

no debe considerarse necesariamente como un “ruido” o un error en el conjunto de datos, sino como una señal valiosa que refleja la ambigüedad real del lenguaje humano.

### 2.10.1. Aprendizaje con Desacuerdo (Learning with Disagreement)

Para gestionar esta multiplicidad de opiniones en la fase de entrenamiento de los modelos, el estado del arte ha evolucionado desde el consenso estricto hacia el **Aprendizaje con Desacuerdo** (*Learning with Disagreement* o *LeWiDi*) [51]. A nivel teórico, el tratamiento de las etiquetas asignadas por múltiples anotadores se aborda mediante distintas estrategias:

- **Hard Labels (Etiquetas rígidas o por Consenso)**: Consiste en fusionar las opiniones de los diferentes evaluadores en una única etiqueta definitiva. La técnica más común es el **voto por mayoría** (si dos de tres anotadores consideran un texto como sexista, la etiqueta final será positiva). En los casos donde existe un empate exacto (por ejemplo, un anotador vota a favor y otro en contra), las estrategias tradicionales optan por descartar la muestra para evitar confundir al modelo. Como alternativa, se pueden aplicar heurísticas de desempate sesgadas hacia la sensibilidad de la tarea: un enfoque pesimista (ante la duda, se cataloga como sexismo) o un enfoque optimista (ante la duda, se absuelve el comentario).
- **Soft Labels (Etiquetas Probabilísticas)**: En lugar de forzar una etiqueta binaria (0 o 1), el modelo se entrena para predecir la distribución de probabilidad de los anotadores. Si tres personas evalúan un texto y dos lo marcan como sexista, la etiqueta *soft* será 0.66. Esto enseña a la red neuronal a medir el grado de polarización o ambigüedad de un mensaje.
- **Desenrollado o Unrolling**: Una estrategia de aumento de datos que evita la pérdida de información de las minorías. Consiste en duplicar el registro original tantas veces como anotadores lo hayan evaluado, asignando a cada copia la etiqueta individual de un anotador diferente. De este modo, la red neuronal se expone a todas las perspectivas humanas sin forzar un consenso matemático.

### 2.10.2. Perspectivismo en Inteligencia Artificial

El perspectivismo (*Data Perspectivism*) [52] es una corriente reciente en la inteligencia artificial que sostiene que las características demográficas de los anotadores (como su género, edad, raza o nivel educativo) condicionan radicalmente sus respuestas. En la detección del sexismo, por ejemplo, la perspectiva de una mujer y la de un hombre frente a un mismo texto pueden diferir significativamente debido a sus vivencias estructurales.



Para capturar y representar algorítmicamente estas visiones del mundo, la literatura propone distintas metodologías de modelado dependiendo de la naturaleza de la arquitectura utilizada:

- **Segmentación demográfica para modelos discriminativos *Transformers*:** Consiste en dividir y filtrar el conjunto de datos de entrenamiento basándose exclusivamente en el perfil demográfico de los anotadores (por ejemplo, filtrando solo los votos emitidos por mujeres o solo por hombres). A partir de estos subconjuntos, se entrenan modelos discriminativos independientes. Esto da como resultado arquitecturas “expertas” que han interiorizado la sensibilidad y los sesgos propios de un único grupo demográfico.
- **Asignación de roles para Modelos Generativos (*LLMs*):** Los grandes modelos de lenguaje han interiorizado múltiples perspectivas durante su preentrenamiento masivo. Para evocar una visión demográfica específica sin necesidad de reentrenar la red, se utiliza la técnica de *Persona Prompting* (ingeniería de instrucciones basada en roles) [53]. Al inyectar directrices explícitas en el *prompt* del sistema (instruyendo al modelo para que “actúe como un hombre” o “actúe como una mujer”), se condiciona el espacio latente de la red para que su inferencia (*Zero-Shot* o *Few-Shot*) adopte el sesgo cognitivo de dicho perfil demográfico.

## 2.11. Medidas de Evaluación

Para cuantificar de manera rigurosa y objetiva el rendimiento de los modelos desarrollados en este trabajo, se utilizan métricas estándar derivadas del aprendizaje automático. La elección de una métrica adecuada es un paso crítico en el diseño experimental, ya que no todas las medidas son igualmente válidas en todos los contextos. En tareas sensibles como la detección del discurso de odio o el sexismo, donde los conjuntos de datos suelen presentar un fuerte desbalanceo (con una presencia mayoritaria de comentarios no sexistas frente a los sexistas), basarse en métricas simples puede ofrecer una falsa sensación de eficacia algorítmica. Por ello, es necesario definir medidas específicas adaptadas a la topología de la tarea: clasificación binaria, multietiqueta o evaluación con desacuerdo.

### 2.11.1. Accuracy, Precisión, Exhaustividad y *F1-Score*

En las tareas de clasificación binaria (como determinar si un contenido es o no es sexista), la evaluación parte de la matriz de confusión, la cual clasifica las predicciones en Verdaderos Positivos (*TP*), Verdaderos Negativos (*TN*), Falsos Positivos (*FP*) y Falsos Negativos (*FN*). A partir de estos valores se derivan las siguientes métricas teóricas [54]:

- **Exactitud (*Accuracy*)**: Mide el porcentaje total de predicciones correctas sobre el total de casos evaluados. Aunque es intuitiva, resulta engañosa en conjuntos de datos desbalanceados.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (2.2)$$

- **Precisión (*Precision*)**: Evalúa la proporción de identificaciones positivas que fueron realmente correctas. Es fundamental cuando el coste de un Falso Positivo es inaceptable.

$$Precision = \frac{TP}{TP + FP} \quad (2.3)$$

- **Exhaustividad (*Recall*)**: Mide la proporción de positivos reales que el modelo logró identificar correctamente. En la detección de discursos de odio, un alto *recall* es vital para garantizar que ningún comentario dañino (Falso Negativo) pase desapercibido.

$$Recall = \frac{TP}{TP + FN} \quad (2.4)$$

- **F1-Score**: Representa la media armónica entre la precisión y el *recall*. Es la métrica estándar por excelencia en el PLN para lidiar con el desbalanceo de clases, ya que penaliza fuertemente a los modelos que favorecen una métrica a costa de destruir la otra.

$$F1-score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (2.5)$$

### 2.11.2. Evaluación en clasificación multietiqueta

En el caso de clasificación multietiqueta (donde una misma instancia puede pertenecer a múltiples categorías simultáneamente), la evaluación matemática se vuelve más compleja. Para abordar este problema se utilizan las siguientes variantes [55]:

- **F1 Macro (*Macro-F1*)**: Calcula el *F1-Score* de manera independiente para cada clase o etiqueta y, posteriormente, extrae la media aritmética no ponderada. Esta aproximación otorga exactamente el mismo peso a todas las clases, independientemente de su frecuencia, lo que la hace ideal para evaluar si el modelo es capaz de detectar las categorías minoritarias.

$$Macro\ F1 = \frac{1}{C} \sum_{c=1}^C F1_c \quad (2.6)$$

Donde  $C$  es el número total de clases posibles.

- **F1 Micro (*Micro-F1*)**: Agrega globalmente las contribuciones de todas las clases sumando los verdaderos positivos, falsos negativos y falsos positivos antes de calcular

la métrica. Está fuertemente influenciada por las clases mayoritarias.

- **Exact Match Ratio (Tasa de coincidencia exacta)**: Es la métrica multietiqueta más estricta. Un acierto solo se contabiliza si el conjunto de etiquetas predichas por el modelo coincide de forma absoluta e idéntica con el conjunto de etiquetas reales de la instancia.

$$Exact\ Match = \frac{1}{N} \sum_{i=1}^N I(Y_i = \hat{Y}_i) \quad (2.7)$$

Donde  $I$  es la función indicadora que devuelve 1 si los conjuntos son idénticos y 0 en caso contrario.

### 2.11.3. Métricas para evaluación con desacuerdo

Cuando se abandona el paradigma del consenso absoluto y se entrena a los modelos utilizando *Soft Labels*, las métricas tradicionales basadas en coincidencias binarias dejan de ser aplicables. En estos escenarios, el objetivo de la evaluación es medir qué tan bien se alinea la distribución de probabilidad generada por la red neuronal con la distribución real de los anotadores humanos.

Para ello, la métrica estándar es la **Entropía Cruzada (*Cross-Entropy*)**, que penaliza matemáticamente las divergencias entre la predicción del modelo ( $\hat{y}$ ) y la opinión humana real ( $y$ ).

$$CE(y, \hat{y}) = - \sum_{c=1}^C y_c \log(\hat{y}_c) \quad (2.8)$$

En el marco de competiciones recientes de análisis de subjetividad (como la tarea *EXIST*), esta divergencia suele estandarizarse mediante la medida **ICM (*Information Contrast Measure*)** [56]. El *ICM* cuantifica la calidad de la información capturada por el modelo frente a un modelo base o aleatorio. Si el modelo logra predecir con exactitud la incertidumbre y el grado de polarización de los anotadores (por ejemplo, prediciendo un 60 % de probabilidad de sexismo en un mensaje altamente debatido), el *ICM* recompensa esta flexibilidad algorítmica, validando que la máquina ha comprendido la ambigüedad inherente al lenguaje natural.

## 2.12. Tecnologías y Recursos Utilizados

El desarrollo de este proyecto se ha llevado a cabo utilizando un conjunto de herramientas y bibliotecas de código abierto ampliamente integradas en el ámbito de la inteligencia

artificial, el PLN y la visión por computador. Mas que una simple enumeración, estas tecnologías conformar un ecosistema cohesionado que ha permitido desde el preprocesamiento de los datos hasta el despliegue de modelos masivos.



Figura 2.10: Tecnologías usadas para el desarrollo del TFM

Como se puede observar en la [Figura 2.10](#), las principales herramientas utilizadas han sido las siguientes:

- **Python:** Lenguaje de programación principal de todo el proyecto, elegido por su flexibilidad, su inmenso ecosistema de librerías científicas y su estandarización absoluta en la comunidad de *Data Science* y el aprendizaje automático.
- **Google Colab Pro y Jupyter Notebook:** Como entorno de desarrollo interactivo se han utilizado cuadernos de *Jupyter* a través de la plataforma *Google Colab*. En concreto, se ha hecho uso de recursos avanzados (acelerados de hardware como las *GPU NVIDIA L4*) para llevar a cabo el entrenamiento de modelos masivos. En total se han consumido unas 300 unidades informáticas.
- **PyTorch:** *Framework* de *Deep Learning* que ha servido como núcleo matemático para construir, entrenar y evaluar todos los modelos basados en *Transformers* implementados en la fase empírica.
- **Ecosistema Hugging Face:** Plataforma y conjunto de bibliotecas clave en el proyecto. Se ha utilizado para importar modelos preentrenados (*Mistral*, *LLaMA*, *Gemma*, *ConvNeXt*, etc.), aplicar técnicas de *PEFT* mediante inserción de matrices de bajo rango, ejecutar optimizadores supervisados como *SFTTrainer*, y formatear eficientemente los conjuntos de datos.
- **BitsAndBytes:** Librería fundamental de optimización de hardware que ha permitido

aplicar la técnica de cuantización *QLoRA*, reduciendo los pesos de los *LLMs* a una precisión de 4 bits para que pudieran ser procesados en la memoria de la tarjeta gráfica disponible.

- ***Decord* y *OpenCV (cv2)***: Bibliotecas especializadas en el procesamiento audiovisual. Han sido fundamentales para el tratamiento de la modalidad de vídeo, permitiendo una lectura optimizada de los archivos multimedia y la extracción precisa de los fotogramas clave (*frames*) requeridos por los modelos visuales.
- ***Scikit-Learn* y *Evaluate***: Herramientas de análisis de datos utilizadas en la fase de evaluación para computar métricas complejas de rendimiento (*F1-Score*, *Accuracy*, matrices de confusión) sobre los conjuntos de prueba estáticos.
- ***Google Drive***: Sistema de almacenamiento en la nube utilizado como repositorio de datos centralizado. Ha servido para alojar, gestionar y sincronizar todos los conjuntos de datos (*datasets*), los pesos de los modelos guardados y los archivos generados durante la experimentación, permitiendo una integración y un acceso director desde los entornos de *Google Colab*.
- ***GitHub***: Plataforma de desarrollo colaborativo y control de versiones. Se ha empleado para estructurar de forma metódica y segura el proyecto mediante repositorios independientes: uno destinado a la centralización del código fuente y los cuadernos de experimentación <sup>1</sup>, y otro para el control de versiones de la presente memoria <sup>2</sup>.
- ***LaTeX***: Sistema de composición de textos utilizado para la redacción, estructuración y maquetación de esta memoria. Su elección garantiza un formato académico riguroso, así como una gestión impecable de las referencias bibliográficas y la representación de fórmulas matemáticas.

A modo de conclusión, en este capítulo se han establecido los fundamentos teóricos necesarios para comprender los enfoques tecnológicos implementados a lo largo del proyecto. Esta base conceptual resulta indispensable para justificar las decisiones técnicas, algorítmicas y metodológicas que se abordará en detalle en el próximo capítulo.

---

<sup>1</sup><https://github.com/visumania/DeepSexist>

<sup>2</sup><https://github.com/visumania/MemoriaTFM>

# Metodología, Experimentación y Resultados

---

### Sugerencia

Como siempre, al comenzar un capítulo y antes de empezar con la primera sección, se debe escribir un párrafo introductorio para explicar lo que el lector se va a encontrar en este capítulo.

## 3.1. Participación en una tarea (si es el caso)

Si el trabajo se ha realizado en el contexto de una tarea como *IberLEF*, *CLEF*, *SemEval*, etc., esta sección debe comenzar con una descripción detallada de dicha tarea. Es fundamental que el lector entienda el marco en el que se sitúa el trabajo.

### Sugerencia

**IMPORTANTE:** evitar usar las palabras “competición”, “concurso”, etc. Mejor usar términos como “reto”, “desafío”, etc.

### Sugerencia

Buscar todos los detalles posibles y relevantes de la tarea en su propia página web y describirlos. Debe quedar muy claro en qué consiste la tarea, su utilidad, su campo de aplicación, el interés que tiene, el número de ediciones, quién la organiza, el workshop en el que está incluida, etc. No escatimar palabras.

Se puede introducir esta parte con un párrafo como el siguiente:

*Este trabajo se ha desarrollado en el marco de la tarea [NombreTarea], organizada en el contexto del workshop [NombreWorkshop], celebrado en [Año]. La tarea tiene como objetivo principal...*

Es recomendable incluir también información como:

- Número de equipos participantes.

- Descripción de los datos proporcionados.
- Métricas de evaluación empleadas.
- Categorías o sub-tareas existentes.

Si habéis conseguido una buena posición, es importante destacarlo con claridad:

*Nuestro sistema obtuvo el segundo mejor resultado en la métrica F1 para la sub-tarea de clasificación multicategoría, situándose por encima de la media de los participantes.*

Y si el trabajo ha dado lugar a un artículo científico, también se puede mencionar:

*Como parte de la participación, se redactó un artículo científico titulado “Título del artículo”, que fue aceptado en el workshop [Nombre del workshop], y publicado en las actas del evento.*

#### Sugerencia

La tarea es un medio para enmarcar el trabajo, pero no el objetivo final del TFG/TFM. El foco debe estar en el aprendizaje, la experimentación, la propuesta y evaluación de metodologías o enfoques propios, y en la comprensión profunda del problema que se aborda.

## 3.2. Descripción General de la Metodología

En esta sección se debe mostrar una visión global y estructurada de la metodología y el marco de experimentación seguido para el desarrollo del trabajo. El objetivo es que el lector comprenda los distintos pasos y fases por los que se ha pasado para construir y evaluar los modelos y, en consecuencia, para resolver la tarea.

#### Sugerencia

Enumeración y descripción breve de los pasos principales en el proceso de entrenamiento del modelo: preprocesamiento, selección o definición del modelo, ajuste de hiperparámetros, entrenamiento, evaluación, etc.

Una posible redacción introductoria podría ser:

*El marco de experimentación se ha dividido en varias fases: preprocesamiento de los datos, diseño y entrenamiento de los modelos, y evaluación de su rendimiento. En esta sección se ofrece una visión general del proceso completo, que será detallado en las secciones posteriores.*

### Ejemplo de estructura de pasos

- Preprocesamiento de datos
- Selección o definición del modelo base
- Ajuste de hiperparámetros
- Entrenamiento del modelo
- Evaluación y análisis de resultados

#### Sugerencia

Incluir un gráfico o diagrama de flujo que visualice la secuencia completa de pasos y procesos seguidos. Este elemento servirá como referencia visual para complementar las descripciones textuales

La [Figura 3.1](#) muestra el pipeline del marco de experimentación seguido en este trabajo.

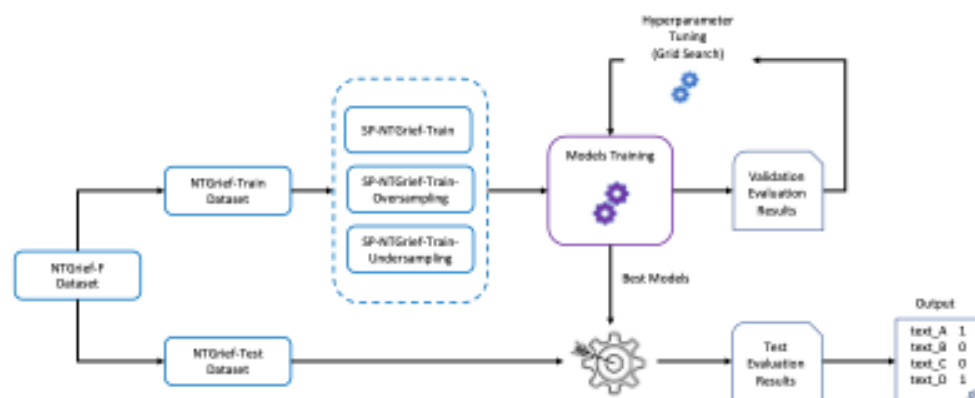


Figura 3.1: Pipeline general del marco de experimentación.

### 3.3. Preparación de los Datos

En esta sección se debe explicar detalladamente cómo se han preparado los datos antes de ser utilizados por los modelos. Esto incluye su exploración inicial, transformación, limpieza, codificación y división en subconjuntos.



### 3.3.1. Descripción de los Datos

#### Sugerencia

Describir los datos proporcionados por la tarea: formato, número de instancias, tipo de anotaciones, etc. Incluir ejemplos reales. Se recomienda acompañar con gráficas exploratorias (por ejemplo, palabras más frecuentes por clase, distribución de etiquetas, tamaños de texto, etc.).

*El dataset proporcionado por la organización de la tarea está compuesto por 3500 memes anotados con una etiqueta de acuerdo o desacuerdo. Además, incluye su texto asociado. El dataset tiene la información de todos los anotadores que han participado en el proceso de anotación...*

*En la Figura "Tal" se muestran las palabras más frecuentes en los textos etiquetados como "sexista".*

### 3.3.2. Balanceo de Datos

#### Sugerencia

Si el dataset original está desbalanceado, indicar qué técnicas se han aplicado para corregirlo (por ejemplo, backtranslation, sobre-muestreo, sub-muestreo...). Especificar qué modelos o herramientas se han usado y cómo ha quedado el dataset final tras el balanceo.

*Dado que la clase "sexista" estaba poco representada (17% del total), se aplicó una estrategia de augmentación mediante backtranslation con el modelo Helsinki-NLP/opus-mt-en-es<sup>1</sup>. Tras el proceso, el dataset aumentó a 5100 instancias.*

#### Sugerencia

**IMPORTANTE:** poner notas al pie es muy fácil en Latex con el entorno footnote. Se utiliza mucho para hacer referencias a direcciones URL cuando no se ponen como referencias, como en el caso del párrafo anterior

<sup>1</sup><https://huggingface.co/Helsinki-NLP/opus-mt-en-es>

### 3.3.3. Limpieza y Normalización

#### Sugerencia

Describir los pasos de limpieza y normalización aplicados al texto, como la conversión a minúsculas, eliminación de signos de puntuación, tratamiento de emojis, corrección de caracteres erróneos, etc. También puede mencionarse el paso de json a csv o la transformación de estructuras.

*Se eliminaron los caracteres especiales no alfabéticos, se convirtieron los textos a minúsculas y se estandarizaron emoticonos comunes. El conjunto original estaba en formato JSON y fue convertido a CSV para facilitar el procesamiento posterior.*

### 3.3.4. Tokenización y Codificación

#### Sugerencia

Explicar cómo se han tokenizado los textos y cómo se convierten en índices o vectores de entrada. Se recomienda incluir un ejemplo sencillo mostrando los tokens generados y sus códigos (input\_ids).

*Para tokenizar los textos se utilizaron los tokenizadores correspondientes a cada modelo. Por ejemplo, el texto “I totally agree” se tokeniza como [“i”, “totally”, “agree”] y se codifica como [1045, totally\_id, agree\_id].*

### 3.3.5. División de Datos

#### Sugerencia

Describir cómo se han dividido los datos en conjuntos de entrenamiento, validación y test. Indicar proporciones y criterios. Acompañar con gráficas de barras o de otro tipo que muestren visualmente las proporciones y el equilibrio de clases en cada subconjunto.

*Los datos se dividieron en 80 % para entrenamiento, 20 % para validación y 20 % para test. Se garantizó el equilibrio de clases en cada subconjunto mediante muestreo estratificado.*

## 3.4. Selección de los Modelos

En esta sección se debe justificar la elección de los modelos empleados para resolver la tarea. Es importante mencionar tanto los modelos que finalmente se han utilizado como aquellos que fueron considerados durante el análisis inicial.

### Sugerencia

Listar los modelos de Transformers evaluados (como *BERT*, *RoBERTa*, *GPT*, *LLaMA*, *Mistral*, etc.) y explicar brevemente las razones por las que se seleccionaron o descartaron. Es obligatorio referenciar los modelos en la bibliografía o mediante notas a pie de página.

*Durante la fase de estudio de la tarea se evaluaron varios modelos preentrenados de Transformers. Entre ellos, destacaron bert-base-uncased, roberta-base y mistral-7b-instruct.*

*La elección final del modelo roberta-base se basó en su buen rendimiento en tareas similares y en la disponibilidad de recursos computacionales adecuados.*

*También se consideró el uso de modelos de mayor tamaño, como LLaMA 2 13B, pero fueron descartados por las limitaciones de memoria en los entornos disponibles.*

## 3.5. Ajuste de Hiperparámetros

En esta sección se debe detallar el proceso seguido para ajustar los hiperparámetros del modelo. Es importante justificar las decisiones tomadas y explicar los métodos utilizados para optimizar el rendimiento del sistema.

### 3.5.1. Métodos de Búsqueda

### Sugerencia

Describir los métodos utilizados para explorar combinaciones de hiperparámetros: búsqueda exhaustiva (*grid search*), búsqueda aleatoria (*random search*), frameworks como *Optuna*, *Wandb*, etc.

*Para el ajuste de hiperparámetros se utilizó una estrategia de búsqueda aleatoria empleando el framework Optuna, que permitió explorar eficientemente combinaciones relevantes sin necesidad de evaluar todas las posibles.*

### 3.5.2. Hiperparámetros Evaluados

#### Sugerencia

Enumerar los hiperparámetros que se han ajustado y especificar el espacio de búsqueda de cada uno.

*Se ajustaron los siguientes hiperparámetros: tasa de aprendizaje (`learning_rate`) en el rango  $[1e-5, 5e-5]$ , número de épocas entre 2 y 6, tamaño de batch entre 8 y 32, etc.*

#### Sugerencia

En lugar de escribir los valores en un párrafo, es conveniente hacer una tabla con el espacio de búsqueda

### 3.5.3. Tamaño del Conjunto de Datos Utilizado

#### Sugerencia

Indicar si se ha utilizado el conjunto completo de entrenamiento o solo una partición para la búsqueda de hiperparámetros. Justificar la decisión.

*Dado el coste computacional del ajuste, se utilizó el 30 % del conjunto de entrenamiento para la búsqueda de hiperparámetros. Se mantuvo una distribución estratificada para preservar el equilibrio entre clases.*

### 3.5.4. Criterios de Selección

#### Sugerencia

Explicar qué métrica se ha utilizado para determinar la combinación de hiperparámetros óptima (por ejemplo, accuracy, F1-score, etc.), y en qué conjunto se evaluó.

*La selección del mejor conjunto de hiperparámetros se basó en la F1-score macro obtenida en el conjunto de validación. Esta métrica fue priorizada al tratarse de un problema con clases desbalanceadas.*

## 3.6. Entrenamiento de los Modelos

Esta sección debe detallar cómo habéis realizado el entrenamiento de los modelos seleccionados, incluyendo las configuraciones, baselines (si se han utilizado) y el seguimiento del

proceso de entrenamiento.

### 3.6.1. Baselines

#### Sugerencia

Si se ha utilizado un sistema baseline, describirlo brevemente: tipo de modelo, hiperparámetros empleados, resultados obtenidos. Es importante establecerlo como referencia para evaluar las mejoras del modelo final.

*Como baseline se utilizó un clasificador ... Este sistema alcanzó una precisión del 68 % en el dataset de test. Sirvió como punto de comparación para evaluar la mejora introducida por los modelos desarrollados en este trabajo.*

### 3.6.2. Configuración de Entrenamiento

#### Sugerencia

Detallar la configuración empleada: librerías utilizadas (por ejemplo, **Trainer** de HuggingFace), número de épocas, valores finales de hiperparámetros, optimizador, regularización (dropout), etc. Indicar también la métrica usada para guardar el mejor modelo.

*El entrenamiento se llevó a cabo con la clase **Trainer** de la librería **transformers** de HuggingFace. Se utilizó el optimizador **AdamW** con una tasa de aprendizaje de  $2 \cdot 10^{-5}$ , un tamaño de batch de 16 y dropout del 0.1. El entrenamiento se realizó durante 4 épocas, y se guardó el modelo con mejor F1-score macro sobre el conjunto de validación.*

#### Sugerencia

Si habéis hecho optimización de hiperparámetros, aquí no debéis repetir el punto “Ajuste de Hiperparámetros”. Anteriormente habéis descrito la estrategia de búsqueda de hiperparámetros, y en esta sección debéis detallar otros aspectos, especialmente los mejores valores de hiperparámetros obtenidos para cada modelo, etc.

### 3.6.3. Monitorización del Entrenamiento

#### Sugerencia

Describir cómo se ha monitorizado el entrenamiento: uso de callbacks, logs, métricas registradas, visualización en herramientas como TensorBoard o la consola de HuggingFace. Incluir capturas o gráficos si es posible.

*Se empleó la integración de **Trainer** con **TensorBoard** para monitorizar en tiempo real la evolución de la pérdida y la métrica de evaluación. Además, se activó el callback **EarlyStoppingCallback** para detener el entrenamiento si no había mejora durante 3 épocas consecutivas.*

*En la [Figura 3.2](#) se muestra un ejemplo de la evolución del F1-score durante el entrenamiento.*

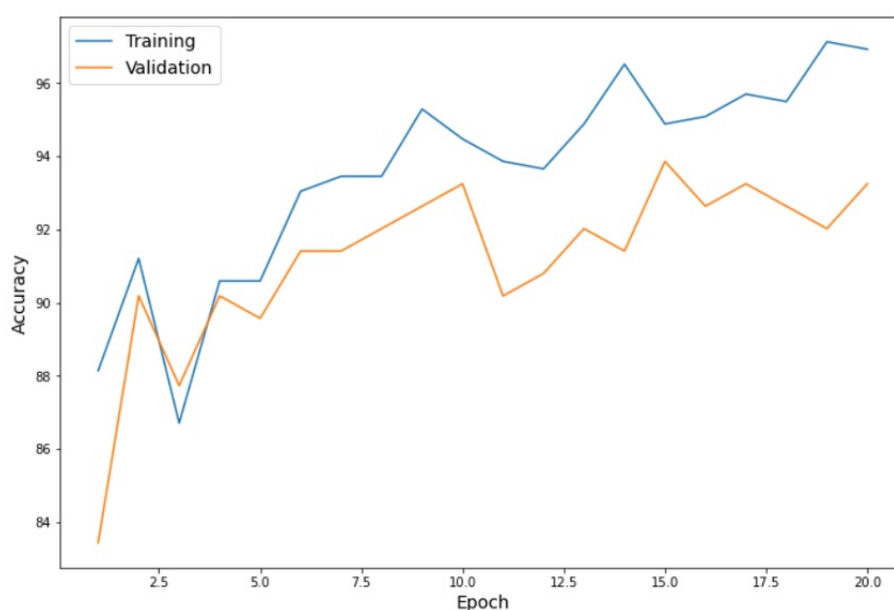


Figura 3.2: Evolución del entrenamiento monitorizada con TensorBoard.

## 3.7. Evaluación y Análisis de Resultados

En esta sección se presentan los resultados obtenidos tras evaluar los modelos entrenados sobre vuestro conjunto de test durante la fase de entrenamiento y mejora de los modelos. Se deben incluir todas las métricas relevantes, gráficos y análisis que permitan interpretar el rendimiento de las distintas estrategias y modelos.

## Sugerencia

Incluye tablas de resultados para todos los modelos evaluados, siguiendo un estilo similar al de los artículos científicos. No olvides añadir títulos claros y destacar los mejores resultados en negrita.

## Sugerencia

Presentar las métricas más relevantes: precisión (accuracy), precisión por clase, recall, F1-score, macro/micro promedio, etc. Utiliza `classification_report` o similares y organiza los resultados en tablas bien formateadas.

En la [Tabla 3.1](#) se presentan los resultados obtenidos por los modelos entrenados sobre el conjunto de test antes del proceso de limpieza. Se incluyen las métricas de F1-score, precisión y recall.

| Modelo       | Precision   | Recall      | F1-score    |
|--------------|-------------|-------------|-------------|
| BERT-base    | 0.76        | 0.78        | 0.75        |
| RoBERTa-base | <b>0.81</b> | <b>0.83</b> | <b>0.80</b> |
| Baseline     | 0.65        | 0.66        | 0.64        |

Tabla 3.1: Resultados de evaluación sobre el conjunto de test.

En la [Tabla 3.2](#) se presentan los resultados obtenidos por los modelos entrenados sobre el conjunto de test desglosados por clase.

Tabla 3.2: Resultados por clase para los modelos evaluados (conjunto de test).

| Modelo       | Clase   | Precision | Recall | F1-score    |
|--------------|---------|-----------|--------|-------------|
| BERT-base    | Clase 1 | 0.81      | 0.79   | 0.80        |
|              | Clase 2 | 0.70      | 0.72   | 0.71        |
|              | Clase 3 | 0.68      | 0.66   | 0.67        |
| RoBERTa-base | Clase 1 | 0.85      | 0.83   | <b>0.84</b> |
|              | Clase 2 | 0.76      | 0.77   | <b>0.76</b> |
|              | Clase 3 | 0.72      | 0.71   | <b>0.71</b> |

## Matrices de Confusión

## Sugerencia

Dibujar las matrices de confusión para los principales modelos, preferiblemente con anotaciones y colores. Explicar lo que se observa: equilibrio en los aciertos, etc.

En la [Figura 3.3](#) se muestra la matriz de confusión del modelo RoBERTa. Se observa que las clases minoritarias son las que peor han sido clasificadas, etc.

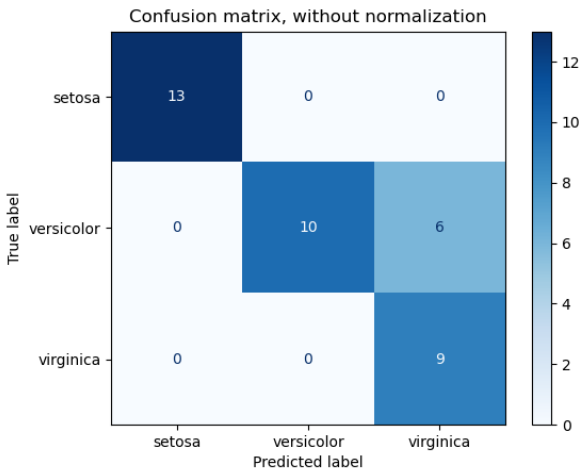


Figura 3.3: Matriz de confusión para el modelo RoBERTa.

En la [Figura 3.4](#) se muestra la matriz de confusión del modelo Llama con las 6 etiquetas. Se observa que se ha comportado muy bien con las clases 1 y 2. Sin embargo, la clase 4 ha obtenido muchos errores, etc.

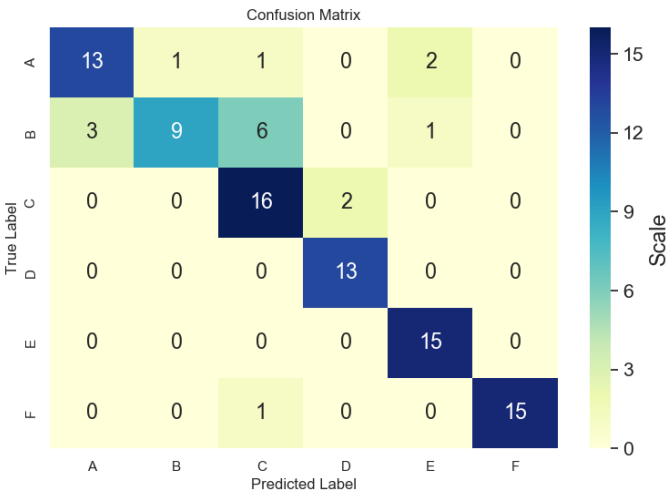


Figura 3.4: Matriz de confusión para el modelo RoBERTa.



## Curvas ROC y AUC

### Sugerencia

Incluir las curvas ROC para cada modelo. Si el problema lo permite (clasificación binaria), puede añadirse también la curva Precision-Recall. Explicar su interpretación y qué significan las áreas bajo las curvas (AUC).

La [Figura 3.5](#) muestra las curvas ROC de los modelos evaluados. El modelo RoBERTa alcanza el mayor valor de AUC, superando el 0.90.

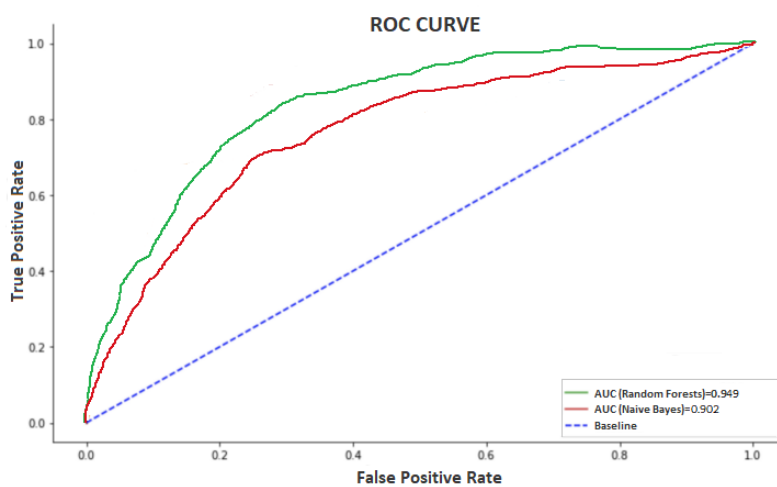


Figura 3.5: Curvas ROC de los modelos evaluados.

## Otras Gráficas Relevantes

### Sugerencia

Se pueden añadir otras visualizaciones complementarias: evolución del entrenamiento, análisis de errores, importancia de tokens o atención, etc.

## 3.8. Análisis de Errores

Esta sección tiene como objetivo destallar algunos de los errores cometidos por el sistema, buscando patrones o causas que expliquen los fallos del modelo. No se trata solo de cuantificar los errores, sino de interpretarlos.

**Sugerencia**

El análisis de errores debe centrarse en identificar los casos en los que el modelo falla sistemáticamente: clases que confunde con frecuencia, tipos de entrada especialmente problemáticos (por longitud, ambigüedad, ruido, etc.), o patrones en los errores.

**Errores más frecuentes**

*Una revisión manual de los errores cometidos por los modelos reveló que gran parte de las instancias mal clasificadas pertenecen a la clase “nombre\_clase1”, que tiende a confundirse con “nombre\_clase2”. Esto sugiere que la frontera entre estas dos clases no está bien definida en algunos casos.*

**Identificación de Errores Comunes****Sugerencia**

Describir los tipos de errores más comunes que cometió el modelo y las posibles razones. Proporcionar ejemplos específicos de entradas mal clasificadas y discutir posibles mejoras o ajustes al modelo o al preprocesamiento.

*Ejemplo 1:*

Texto: “A las mujeres hay que respetarlas porque son lo más bello del mundo”

Etiqueta real: Sexista    Predicción: No sexista

*Comentario: Este mensaje fue clasificado erróneamente como no sexista, probablemente debido a que el clasificador interpreta el lenguaje positivo como respetuoso, sin captar el sesgo implícito de cosificación. Este tipo de error podría corregirse entrenando con ejemplos que incluyan sexismo sutil o benevolente.*

*Ejemplo 2:*

Texto: “Haz lo que quieras, tú siempre sabes más que todos”

Etiqueta real: Sarcástico    Predicción: Literal / Acuerdo

*Comentario: El modelo no detectó el tono irónico y clasificó el mensaje como afirmativo. Es probable que el sarcasmo indirecto no esté bien representado en los datos de entrenamiento.*

### Errores por longitud o tipo de texto

#### Sugerencia

Analiza si hay correlación entre la longitud del texto y el rendimiento del modelo, o si hay ciertos formatos (mensajes muy cortos, uso de emojis, sarcasmo, etc.) que causan más errores.

*Al analizar los errores en función de la longitud de los textos, se observó que los mensajes de menos de 10 palabras presentaban una tasa de error significativamente mayor. Muchos de ellos eran frases irónicas o ambiguas.*

### Reflexiones y posibles mejoras

#### Sugerencia

Cierra esta sección proponiendo mejoras basadas en el análisis de errores.

*Una posible mejora utilizar modelos previamente entrenados sobre detección de ironía o sexismo implícito. También se podrían generar más ejemplos sintéticos, etc..*

## 3.9. Resultados Oficiales de la Tarea (opcional)

Esta sección es opcional y está pensada para aquellos trabajos que hayan participado en una competición oficial. El objetivo es mostrar los resultados enviados durante la fase final de la tarea y contextualizarlos dentro del ranking global.

#### Sugerencia

Indicar cuáles fueron las estrategias que mejores resultados ofrecieron durante la fase de desarrollo y que se seleccionaron para ser enviadas como *runs* oficiales a la competición.

*Durante la fase de desarrollo, se probaron varias configuraciones. La mejor combinación fue la basada en el modelo RoBERTa con aumento de datos mediante backtranslation. Esta estrategia fue seleccionada como uno de los runs oficiales enviados a la competición.*

#### Sugerencia

Describir las características del conjunto de test proporcionado por la competición, si es distinto al de validación. Incluir los resultados oficiales publicados por la organización. Comentar si los resultados fueron similares o no a los esperados.

El conjunto de test oficial incluía 800 ejemplos adicionales, no vistos durante la fase de desarrollo. Los resultados oficiales se muestran en la [Tabla 3.3](#). Aunque nuestro sistema mantuvo un rendimiento estable, el nivel general de los participantes fue más bajo de lo esperado, lo que pone en valor el resultado alcanzado.

| Participante | F1-score    | Ranking  |
|--------------|-------------|----------|
| Equipo A     | 0.83        | 1        |
| Equipo B     | 0.81        | 2        |
| I2C          | <b>0.79</b> | <b>3</b> |
| Media        | 0.65        | –        |

Tabla 3.3: Resultados oficiales publicados por la organización de la tarea.

El sistema presentado quedó en tercera posición, por encima de la media global de todos los participantes, lo que demuestra la solidez de la metodología seguida durante el desarrollo del trabajo.

Publicación de los Resultados

Sugerencia

Si se ha contribuido con un artículo científico a las actas del workshop (por ejemplo, IberLEF o CLEF), se debe mencionar aquí y referenciar correctamente. Esto aporta valor académico al trabajo.

Los resultados obtenidos se recogieron en el artículo científico titulado “Título del artículo”, presentado en el workshop IberLEF 2024, y publicado en las actas del evento.

El artículo puede consultarse en [CEUR-Ws<sup>2</sup>](#), y se ha incluido en un Anexo de esta memoria.

<sup>2</sup><http://ceur-ws.org/Vol-XXX>

# Conclusiones y Trabajo Futuro

Este capítulo debe ofrecer una síntesis clara del trabajo realizado, valorando los resultados obtenidos y proponiendo posibles trabajos para continuar la investigación. También debe justificarse aquí el esfuerzo temporal invertido.

## 4.1. Conclusiones

### Sugerencia

Describir cómo se han alcanzado los objetivos planteados, cuáles han sido los principales hallazgos, sus implicaciones, posibles limitaciones y la contribución del trabajo dentro del área de estudio.

### Consecución de Objetivos

*Los objetivos definidos al inicio del trabajo han sido alcanzados satisfactoriamente. Se ha desarrollado un sistema de clasificación basado en Transformers y se ha evaluado en el marco de una competición oficial con buenos resultados.*

### Resumen de los Hallazgos

*El modelo RoBERTa, combinado con técnicas de aumento de datos, ha logrado una mejora del 25 % en F1-score respecto al baseline. El análisis de errores ha revelado que los errores más comunes están relacionados con el lenguaje irónico y ambiguo.*

### Implicaciones

*Los resultados obtenidos demuestran la viabilidad de usar modelos de lenguaje preentrenados para tareas de clasificación de texto en contextos sensibles como la detección de discursos sesgados o polarizados.*

### Limitaciones

*Una limitación importante ha sido el desequilibrio de clases y la falta de ejemplos anotados con ironía sutil o sexismo benevolente, lo que ha dificultado el entrenamiento del modelo.*

### Contribuciones

*Este trabajo ha contribuido con una metodología reproducible para el tratamiento de datos textuales en español, ha evaluado distintas arquitecturas y ha participado activamente en una tarea competitiva, generando un artículo científico.*

### Consideraciones Éticas

*Es importante tener en cuenta las posibles implicaciones éticas del uso de modelos automáticos para clasificar lenguaje humano, especialmente en contextos como el discurso sexista. Las decisiones automatizadas deben siempre contextualizarse y acompañarse de revisión humana.*

## 4.2. Trabajo Futuro

#### Sugerencia

Proponer líneas de trabajo que den continuidad al proyecto, ya sea en forma de mejoras técnicas, nuevas preguntas de investigación, aplicaciones prácticas o ampliaciones del estudio.

### Extensiones del Estudio

*Una posible extensión sería aplicar el mismo enfoque a otros dominios del lenguaje subjetivo, como la detección de odio o polarización en redes sociales.*

### Investigaciones Adicionales

*Podría investigarse la eficacia de modelos multilingües o adaptativos en tareas con clases ambiguas o poco representadas.*

Aplicaciones Prácticas

*Los resultados podrían utilizarse para desarrollar herramientas de apoyo a la moderación de contenido o de análisis sociolingüístico en medios digitales.*

Mejoras Metodológicas

*Se propone explorar técnicas de aprendizaje activo para mejorar la eficiencia de la anotación de datos y reducir el sesgo en los conjuntos de entrenamiento.*

4.3. Planificación Temporal del Trabajo Realizado

En este apartado se muestra el tiempo empleado en el estudio y resolución de las diferentes partes de nuestro TFG/TFM, detallando así las tareas más complejas y las horas globales empleadas en ellas en la [Tabla 4.1](#).

Tabla 4.1: Planificación temporal del trabajo realizado.

| Tarea                                                                        | Horas      |
|------------------------------------------------------------------------------|------------|
| Estudio de la Tarea <i>Semeval 2024</i>                                      | 5          |
| <b>Aprendizaje de los conceptos necesarios para abordar el TFG:</b>          | <b>70</b>  |
| - Aprendizaje automático, redes neuronales, Transformers                     |            |
| - Transfer Learning, Fine Tuning, PLN, métricas de evaluación                |            |
| <b>Aprendizaje de la tecnología necesaria:</b>                               | <b>80</b>  |
| - PyTorch, modelos de Transfer Learning para NLP                             |            |
| - Librerías como NLTK, modelos multimodales, L <sup>A</sup> T <sub>E</sub> X |            |
| <b>Realización de la tarea de <i>Semeval 2024</i>:</b>                       | <b>100</b> |
| - Descarga y estudio de datasets, tratamiento de datos                       |            |
| - Diseño de experimentos, implementación, entrenamiento                      |            |
| Elaboración del <i>paper</i> y correcciones                                  | 50         |
| Elaboración de la memoria del proyecto                                       | 35         |
| <b>Total</b>                                                                 | <b>340</b> |

En esta tabla se refleja el número de horas que se ha dedicado a cada tarea, siendo el desarrollo de la tarea *Semeval 2024* la que más tiempo nos ha llevado, con una diferencia de 20 horas respecto a la siguiente más costosa.

Se han invertido también unas 70 horas en la etapa de formación, estudio de la teoría y los conceptos necesarios para abordar eficazmente la tarea y alcanzar los objetivos del trabajo. La mayor parte de estos conceptos no se abordan en el plan de estudios habitual, lo que nos llevó a buscar material adicional en fuentes especializadas y recursos online.

Se han empleado unas 80 horas en el aprendizaje de las tecnologías necesarias para desarrollar el proyecto. El desarrollo del software necesario para resolver el problema planteado en la competición ha supuesto unas 100 horas.

Una vez completado el plazo de participación, se elaboró el artículo científico y se revisó, lo que supuso unas 50 horas adicionales. Finalmente, para la elaboración y revisión de la memoria del proyecto se dedicaron unas 35 horas.



# Referencias

---

- [1] Laura Plaza et al. “Overview of exist 2025: Learning with disagreement for sexism identification and characterization in tweets, memes, and tiktok videos”. En: *International Conference of the Cross-Language Evaluation Forum for European Languages*. Springer. 2025, págs. 266-289.
- [2] Lin Tian, Johanne R Trippas y Marian-Andrei Rizoiu. “Mario at EXIST 2025: a simple gateway to effective multilingual sexism detection”. En: *arXiv preprint arXiv:2507.10996* (2025).
- [3] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, 1997.
- [4] Christopher M Bishop. *Pattern recognition and machine learning*. 2006.
- [5] Trevor Hastie, Robert Tibshirani, Jerome Friedman et al. *The elements of statistical learning*. 2009.
- [6] Andrew G Barto. “Reinforcement learning: Connections, surprises, challenges”. En: *AI Magazine* 40.1 (2019), págs. 3-15.
- [7] Min-Ling Zhang y Zhi-Hua Zhou. “A review on multi-label learning algorithms”. En: *IEEE transactions on knowledge and data engineering* 26.8 (2013), págs. 1819-1837.
- [8] Shervin Minaee et al. “Deep learning-based text classification: a comprehensive review”. En: *ACM computing surveys (CSUR)* 54.3 (2021), págs. 1-40.
- [9] Andrej Karpathy et al. “Large-scale video classification with convolutional neural networks”. En: *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*. 2014, págs. 1725-1732.
- [10] Tadas Baltrušaitis, Chaitanya Ahuja y Louis-Philippe Morency. “Multimodal machine learning: A survey and taxonomy”. En: *IEEE transactions on pattern analysis and machine intelligence* 41.2 (2018), págs. 423-443.
- [11] Ian Goodfellow, Yoshua Bengio y Aaron Courville. *Deep Learning*. MIT Press, 2016. URL: <https://www.deeplearningbook.org>.
- [12] Yann LeCun, Yoshua Bengio y Geoffrey Hinton. “Deep learning”. En: *Nature* 521.7553 (2015), págs. 436-444.
- [13] Francois Chollet y Matthew Watson. *Deep learning with Python*. Vol. 2. Manning publications Shelter Island, NY, USA, 2021.
- [14] Lutz Prechelt. “Early stopping-but when?” En: *Neural Networks: Tricks of the trade*. Springer, 2002, págs. 55-69.
- [15] Yoshua Bengio. “Practical recommendations for gradient-based training of deep architectures”. En: *Neural networks: Tricks of the trade: Second edition*. Springer, 2012, págs. 437-478.
- [16] Dominic Masters y Carlo Luschi. “Revisiting small batch training for deep neural networks”. En: *arXiv preprint arXiv:1804.07612* (2018).
- [17] Sebastian Ruder. “An overview of gradient descent optimization algorithms”. En: *arXiv preprint arXiv:1609.04747* (2016).

- [18] Anders Krogh y John Hertz. “A simple weight decay can improve generalization”. En: *Advances in neural information processing systems* 4 (1991).
- [19] Ilya Loshchilov y Frank Hutter. “Decoupled weight decay regularization”. En: *arXiv preprint arXiv:1711.05101* (2017).
- [20] Shai Shalev-Shwartz y Shai Ben-David. *Understanding machine learning: From theory to algorithms*. Cambridge university press, 2014.
- [21] Kevin P Murphy. *Machine learning: a probabilistic perspective*. MIT press, 2012.
- [22] Jinseok Nam et al. “Large-scale multi-label text classification—revisiting neural networks”. En: *Joint european conference on machine learning and knowledge discovery in databases*. Springer. 2014, págs. 437-452.
- [23] Xue Ying. “An overview of overfitting and its solutions”. En: *Journal of physics: Conference series*. Vol. 1168. 2. IOP Publishing. 2019, pág. 022022.
- [24] Nitish Srivastava et al. “Dropout: a simple way to prevent neural networks from overfitting”. En: *The journal of machine learning research* 15.1 (2014), págs. 1929-1958.
- [25] Mohammad Mahdi Bejani y Mehdi Ghatee. “A systematic review on overfitting control in shallow and deep neural networks: MM Bejani, M. Ghatee”. En: *Artificial Intelligence Review* 54.8 (2021), págs. 6391-6438.
- [26] James Bergstra y Yoshua Bengio. “Random search for hyper-parameter optimization.” En: *Journal of machine learning research* 13.2 (2012).
- [27] Qian Li et al. “A survey on text classification: From traditional to deep learning”. En: *ACM Transactions on Intelligent Systems and Technology (TIST)* 13.2 (2022), págs. 1-41.
- [28] Bonan Min et al. “Recent advances in natural language processing via large pre-trained language models: A survey”. En: *ACM Computing Surveys* 56.2 (2023), págs. 1-40.
- [29] Diksha Khurana et al. “Natural language processing: state of the art, current trends and challenges”. En: *Multimedia tools and applications* 82.3 (2023), págs. 3713-3744.
- [30] Dan Jurafsky, James H Martin et al. *Speech and language processing*. Vol. 3. Pearson London, 2014.
- [31] Ashish Vaswani et al. “Attention is all you need”. En: *Advances in neural information processing systems* 30 (2017).
- [32] Jacob Devlin et al. “Bert: Pre-training of deep bidirectional transformers for language understanding”. En: *arXiv preprint arXiv:1810.04805* (2018).
- [33] Yinhan Liu et al. “Roberta: A robustly optimized bert pretraining approach”. En: *arXiv preprint arXiv:1907.11692* (2019).
- [34] Wayne Xin Zhao et al. “A survey of large language models”. En: *arXiv preprint arXiv:2303.18223* (2023).
- [35] Tim Dettmers et al. “QLoRA: Efficient Finetuning of Quantized LLMs”. En: *arXiv preprint arXiv:2305.14314* (2023).

- [36] Joao Carreira y Andrew Zisserman. “Quo vadis, action recognition? a new model and the kinetics dataset”. En: *proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2017, págs. 6299-6308.
- [37] Alexey Dosovitskiy et al. “An image is worth 16x16 words: Transformers for image recognition at scale”. En: *arXiv preprint arXiv:2010.11929* (2020).
- [38] Ze Liu et al. “Swin transformer: Hierarchical vision transformer using shifted windows”. En: *Proceedings of the IEEE/CVF international conference on computer vision*. 2021, págs. 10012-10022.
- [39] Zhuang Liu et al. “A convnet for the 2020s”. En: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2022, págs. 11976-11986.
- [40] Kaiming He et al. “Deep residual learning for image recognition”. En: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, págs. 770-778.
- [41] Gedas Bertasius, Heng Wang y Lorenzo Torresani. “Is space-time attention all you need for video understanding?” En: *ICML*. Vol. 2. 3. 2021, pág. 4.
- [42] Anurag Arnab et al. “Vivit: A video vision transformer”. En: *Proceedings of the IEEE/CVF international conference on computer vision*. 2021, págs. 6836-6846.
- [43] Zhan Tong et al. “Videomae: Masked autoencoders are data-efficient learners for self-supervised video pre-training”. En: *Advances in neural information processing systems*. Vol. 35. 2022, págs. 10078-10093.
- [44] Sinno Jialin Pan y Qiang Yang. “A survey on transfer learning”. En: *IEEE Transactions on knowledge and data engineering* 22.10 (2009), págs. 1345-1359.
- [45] Xipeng Qiu et al. “Pre-trained models for natural language processing: A survey”. En: *Science China Technological Sciences* 63.10 (2020), págs. 1872-1897.
- [46] Jeremy Howard y Sebastian Ruder. “Universal language model fine-tuning for text classification”. En: *arXiv preprint arXiv:1801.06146* (2018).
- [47] Jesse Dodge et al. “Fine-tuning pretrained language models: Weight initializations, data orders, and early stopping”. En: *arXiv preprint arXiv:2002.06305* (2020).
- [48] Takuya Akiba et al. “Optuna: A next-generation hyperparameter optimization framework”. En: *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 2019, págs. 2623-2631.
- [49] Omer Sagi y Lior Rokach. “Ensemble learning: A survey”. En: *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 8.4 (2018), e1249.
- [50] Tadas Baltrušaitis, Chaitanya Ahuja y Louis-Philippe Morency. “Multimodal machine learning: A survey and taxonomy”. En: *IEEE transactions on pattern analysis and machine intelligence* 41.2 (2018), págs. 423-443.
- [51] Barbara Plank. “The “problem” of human label variation: On ground truth in data, modeling and evaluation”. En: *arXiv preprint arXiv:2211.02570* (2022).
- [52] Valerio Basile et al. “We need to consider disagreement in evaluation”. En: *Proceedings of the 1st Workshop on Benchmarking: Past, Present and Future*. 2021, págs. 15-21.
- [53] Ameet Deshpande et al. “Toxicity in chatgpt: Analyzing persona-assigned language models”. En: *arXiv preprint arXiv:2304.05335* (2023).

- 
- [54] Marina Sokolova y Guy Lapalme. “A systematic analysis of performance measures for classification tasks”. En: *Information processing & management* 45.4 (2009), págs. 427-437.
  - [55] Grigorios Tsoumakas y Ioannis Katakis. “Multi-label classification: An overview”. En: *International Journal of Data Warehousing and Mining (IJDWM)* 3.3 (2007), págs. 1-13.
  - [56] Flor Miriam Plaza-del-Arco et al. “Overview of EXIST 2023: learning with disagreement for sexism identification and characterization”. En: *Experimental IR Meets Multilinguality, Multimodality, and Interaction: 14th International Conference of the CLEF Association, CLEF 2023*. 2023.

### A.1. Aplicación Web para la Visualización y Prueba del Modelo

Como parte complementaria del trabajo, se ha desarrollado una aplicación web que permite cargar el modelo entrenado y utilizarlo para clasificar nuevas instancias de texto.

Esta aplicación proporciona una interfaz sencilla e intuitiva para que cualquier usuario pueda interactuar con el modelo sin necesidad de conocimientos técnicos.

#### Tecnologías Utilizadas

- **Backend:** Python con `FastAPI`, modelo cargado con `transformers`.
- **Frontend:** HTML, CSS y JavaScript con `Bootstrap`.
- **Despliegue:** Aplicación alojada en *HuggingFace Spaces*.

#### Acceso a la Aplicación

La aplicación puede accederse desde la siguiente URL:

<https://miPagina.es>

#### Manual de Usuario

1. Acceder a la URL anterior desde cualquier navegador.
2. Introducir el texto que se desea clasificar en el campo correspondiente.
3. Pulsar el botón “Clasificar”.
4. Se mostrará la predicción del modelo y, opcionalmente, las probabilidades asociadas a cada clase.

**Ejemplo de uso**

*Texto introducido:* “No me parece bien lo que estás diciendo.” *Salida del modelo:* Clase: Desacuerdo

**Capturas de pantalla**